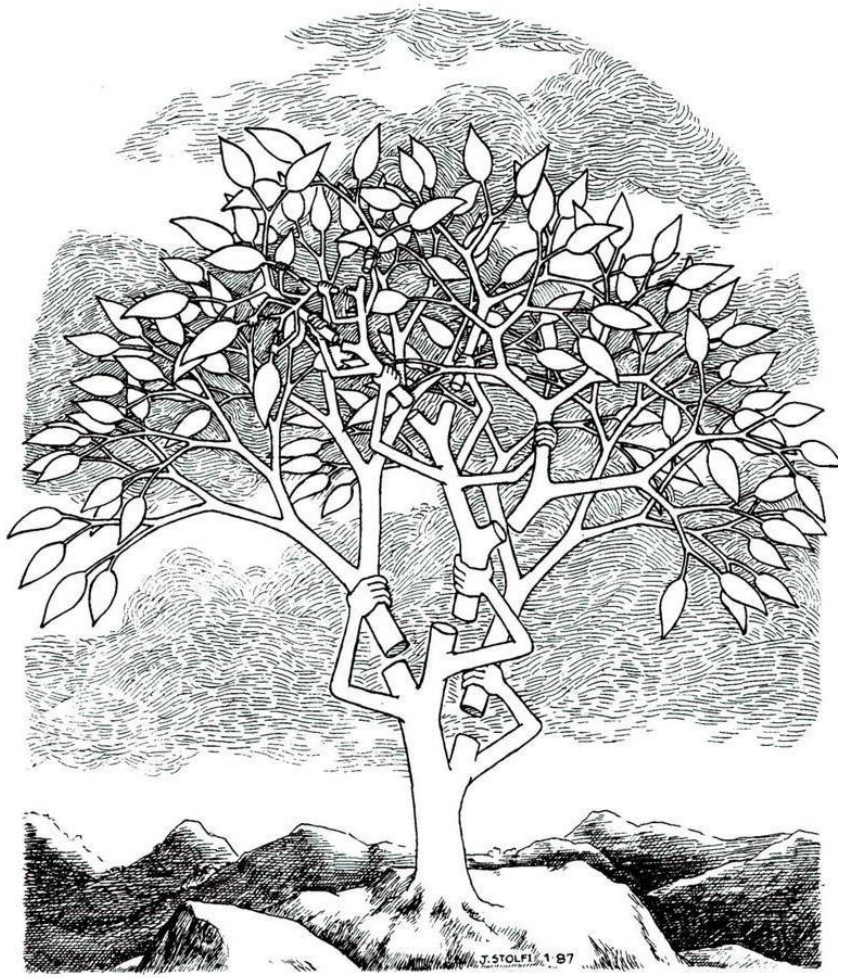


# Árvores

# B

# Árvore B

- Mais de uma chave é armazenada em cada nó.

- Todas as folhas estão no mesmo nível, não importando a ordem de entrada dos dados.

# Árvore B

Aplicações onde existe um número muito grande de dados e o armazenamento do conjunto de chaves não pode ser efetuado na memória principal.



# Árvore B

Torna-se necessária a manutenção da tabela em memória secundária, aumentando significativamente o tempo de acesso de um nó.





# Árvore B

Considere *d* um número natural.

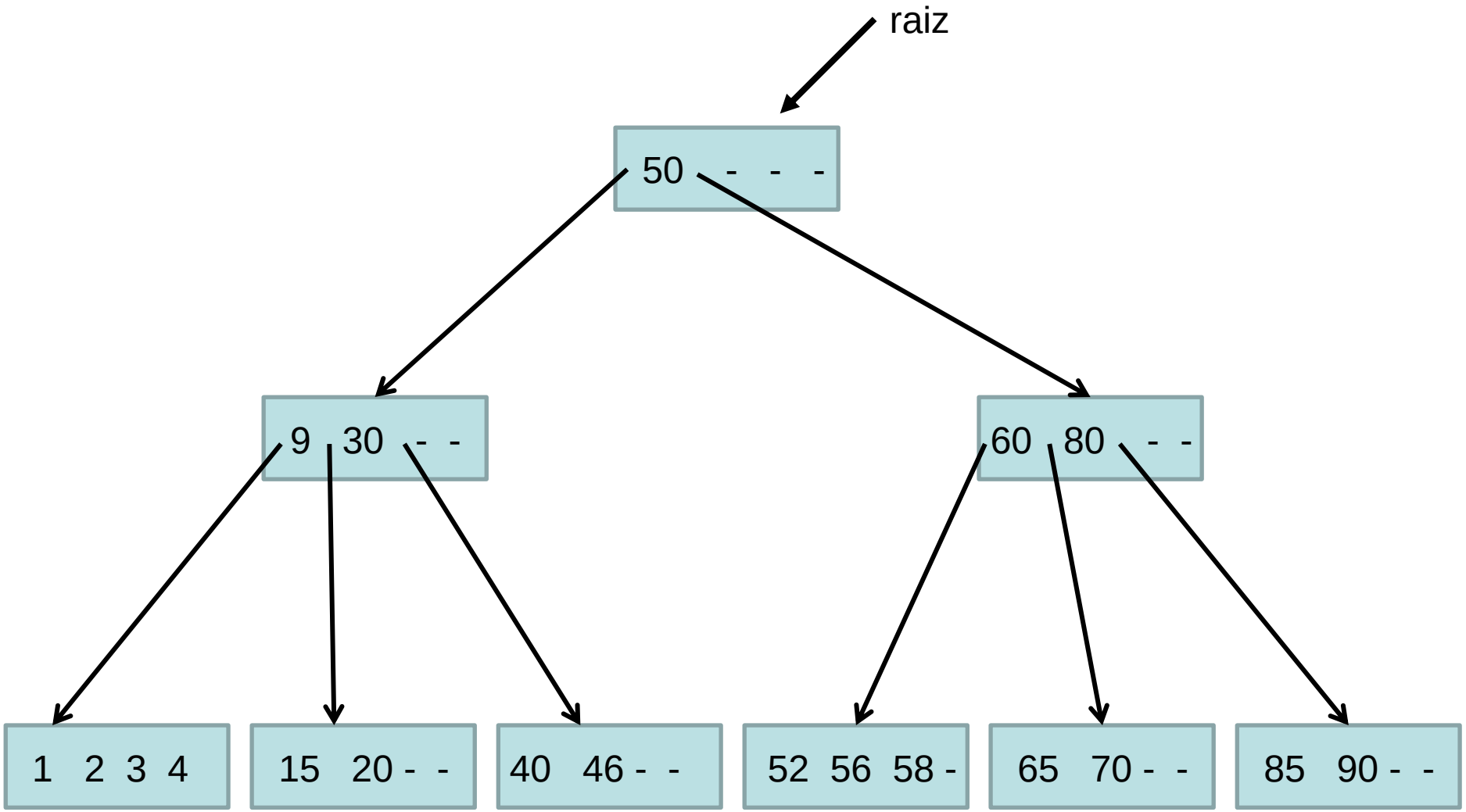
Uma árvore B de ordem *d* é uma árvore que é vazia ou satisfaz as seguintes condições

*(i)* a raiz é uma folha ou tem, no mínimo, dois filhos.

*(ii)* cada nó diferente das folhas e da raiz tem, no mínimo, $d + 1$ filhos

*(iii)* cada nó tem no máximo $2d + 1$ filhos.

# Árvore B de ordem 2



# Árvore B

Um nó de uma árvore B é chamado de **PÁGINA** e apresenta as seguintes propriedades.

*(i)* cada página possui entre *d* e 2*d* chaves, exceto o nó raiz que possui entre 1 e 2*d* chaves.

*(ii)* seja *m* o número de chaves em uma página *P.* Então *P* possui *m* + 1 filhos.

*(iii)* em cada página *P* as chaves estão ordenadas: $s_1, \ldots, s_m$, $d \leq m \leq 2d$, exceto a página raiz onde $1 \leq m \leq 2d$.

# Árvore B

Um nó de uma árvore B é chamado de **PÁGINA** e apresenta as seguintes propriedades.

*(i)* cada página possui entre *d* e 2*d* chaves, exceto o nó raiz que possui entre 1 e 2*d* chaves.

*(ii)* seja *m* o número de chaves em uma página *P.* Então *P* possui *m* + 1 filhos.

*(iii)* em cada página *P* as chaves estão ordenadas: $s_1, \ldots, s_m$, $d \leq m \leq 2d$, exceto a página raiz onde $1 \leq m \leq 2d$.

# Árvore B

Um nó de uma árvore B é chamado de **PÁGINA** e apresenta as seguintes propriedades.

*(i)* cada página possui entre $d$ e $2d$ chaves, exceto o nó raiz que possui entre 1 e $2d$ chaves.

*(ii)* seja $m$ o número de chaves em uma página $P$.
Então $P$ possui $m + 1$ filhos.

*(iii)* Em cada página $P$ as chaves estão ordenadas:
$s_1, \ldots, s_m$, $d \leq m \leq 2d$, exceto a página raiz onde $1 \leq m \leq 2d$.

# Árvore B

*P* possui *m* + 1 ponteiros $p_0,\ldots,p_m$ para os filhos de *P*.

| | | | | |
|---|---|---|---|---|
| $p_0$ | $s_1$ $p_1$ | $s_2$ $p_2$ | ... | $s_m$ $p_m$ |

($s_k$, $p_k$) é uma entrada, $k > 0$
$p_0$ é a entrada, se $k = 0$

# Árvore B

Nas folhas, os ponteiros indicam $\lambda$.

| | | | | |
|---|---|---|---|---|
| $\lambda$ | $s_1$ $\lambda$ | $s_2$ $\lambda$ | ... | $s_m$ $\lambda$ |

# Árvore B

Considere $P$ uma página com $m$ chaves.

Qualquer que seja a chave $y$

- pertencente à página apontada por $p_0$, $y < s_1$

-pertencente à página apontada por $p_k$, $1 \leq k \leq m - 1$, $s_k < y < s_{k+1}$

- pertencente à página apontada por $p_m$, $y > s_m$.

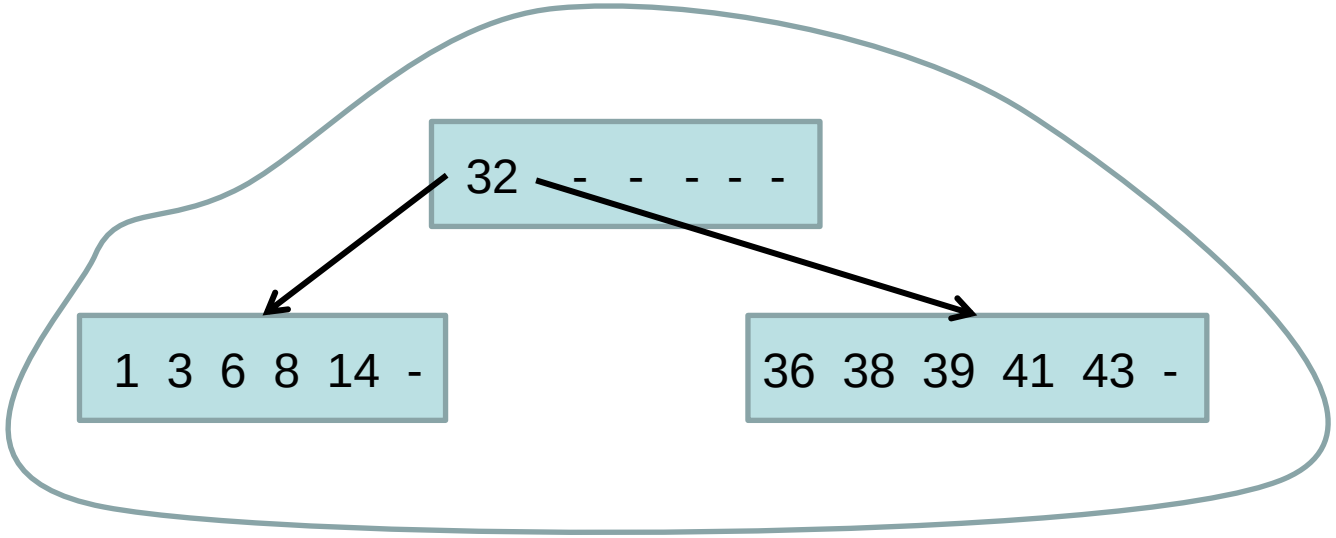
# Exercício

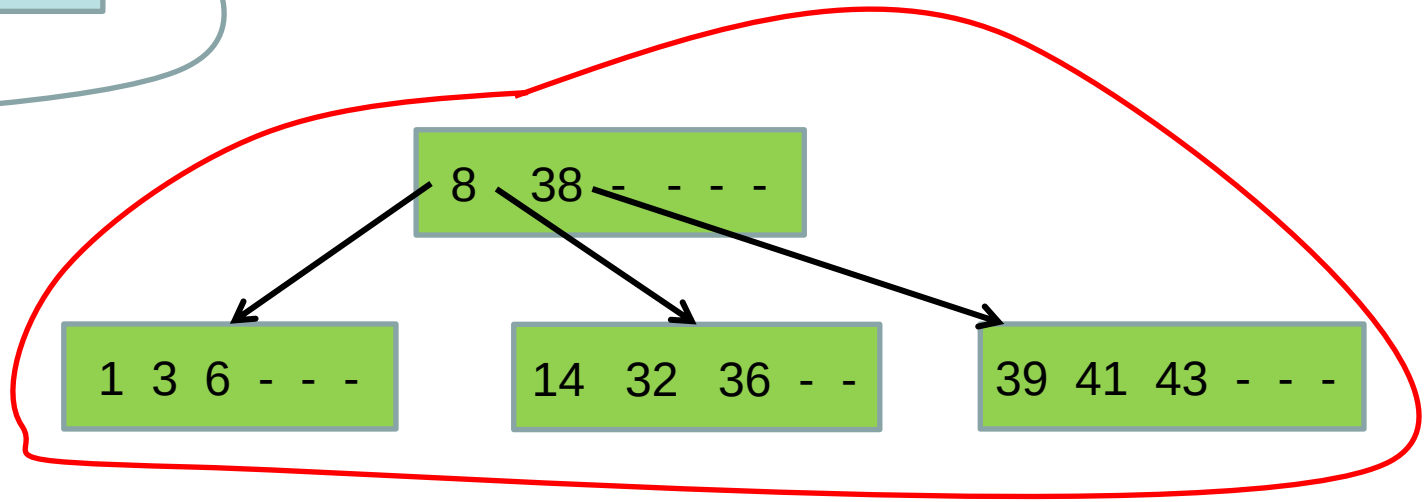
1. Desenhar uma árvore B de ordem 3 que contenha as seguintes chaves:

   1, 3, 6, 8, 14, 32, 36, 38, 39, 41, 43.

# Exercício

1. Desenhar uma árvore B de ordem 3 que contenha as seguintes chaves:
   1, 3, 6, 8, 14, 32, 36, 38, 39, 41, 43.



# Árvore B

## Número Mínimo e Máximo de Páginas

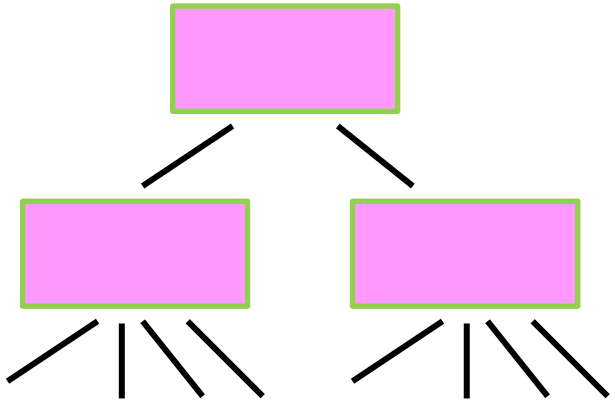
Considere uma árvore B de ordem *d* e altura *h*.

- A raiz possui, no mínimo, 2 filhos.

- Cada página não folha ≠ raiz possui, no mínimo, *d* + 1 filhos

# Árvore B

## Número Mínimo de Páginas



Nível 1: 1 página

Nível 2: 2 páginas

Nível 3: 2(d+1) páginas

Nível $i$: $2(d+1)^{i-2}$ páginas

Nível $h$: $2(d+1)^{h-2}$ páginas

# Árvore B

## Número Mínimo de Páginas

$$P_{MIN} = 1 + 2 [ (d+1)^0 + (d+1)^1 + \ldots + (d+1)^{h-2} ]$$

$$P_{MIN} = 1 + 2/d [ (d+1)^{h-1} - 1], \quad h \geq 1$$

# Árvore B

## Número Máximo de Páginas

$$P_{MAX} = 1 + (2d + 1) + \ldots + (2d+1)^{h-1}$$

$$P_{MAX} = \sum_{k=0}^{h-1} (2d+1)^k$$

$$P_{MAX} = 1/(2d)\ [(2d + 1)^h - 1],\ h \geq 1$$

# Árvore B

## Número Mínimo de Chaves em Árvore B

$$n_{MIN} = 1 + d \{2/d [ (d+1)^{h-1} - 1] \}$$

$$n_{MIN} = 2(d+1)^{h-1} - 1$$

# Árvore B

## Número Máximo de Chaves em Árvore B

$$n_{MAX} = 2d \{1/(2d) [(2d + 1)^h – 1] \}$$

$$n_{MAX} = (2d + 1)^h – 1$$

# Árvore B

## Altura *h* de uma Árvore B

$n_{MIN} = 2(d+1)^{h-1} - 1 \longrightarrow h \leq \log_{d+1}((n+1)/2)$

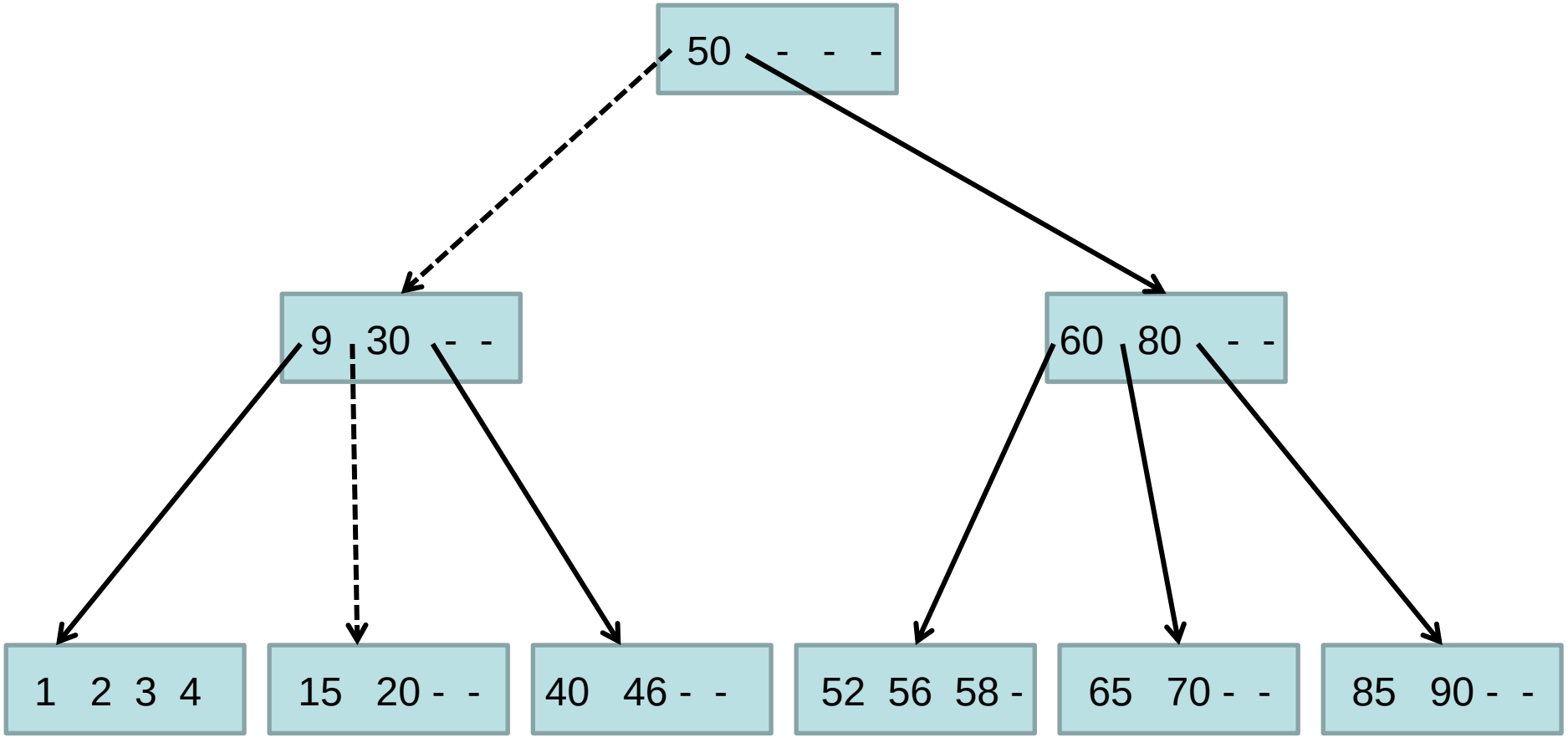
$n_{MAX} = (2d+1)^{h} - 1 \longrightarrow h \geq \log_{2d+1}(n+1)$

$$\log_{2d+1}(n+1) \leq h \leq \log_{d+1}\left(\frac{n+1}{2}\right)$$

**A árvore B é balanceada.**

# Busca em Árvore B

Exemplo: Busca da chave 20 (sucesso), chave 25 (falha)



# Busca em Árvore B

```
Procedimento buscaB (x, pt, f, g)
// f = 1, sucesso; f = 0, falha
// g – posição do nó apontado por pt

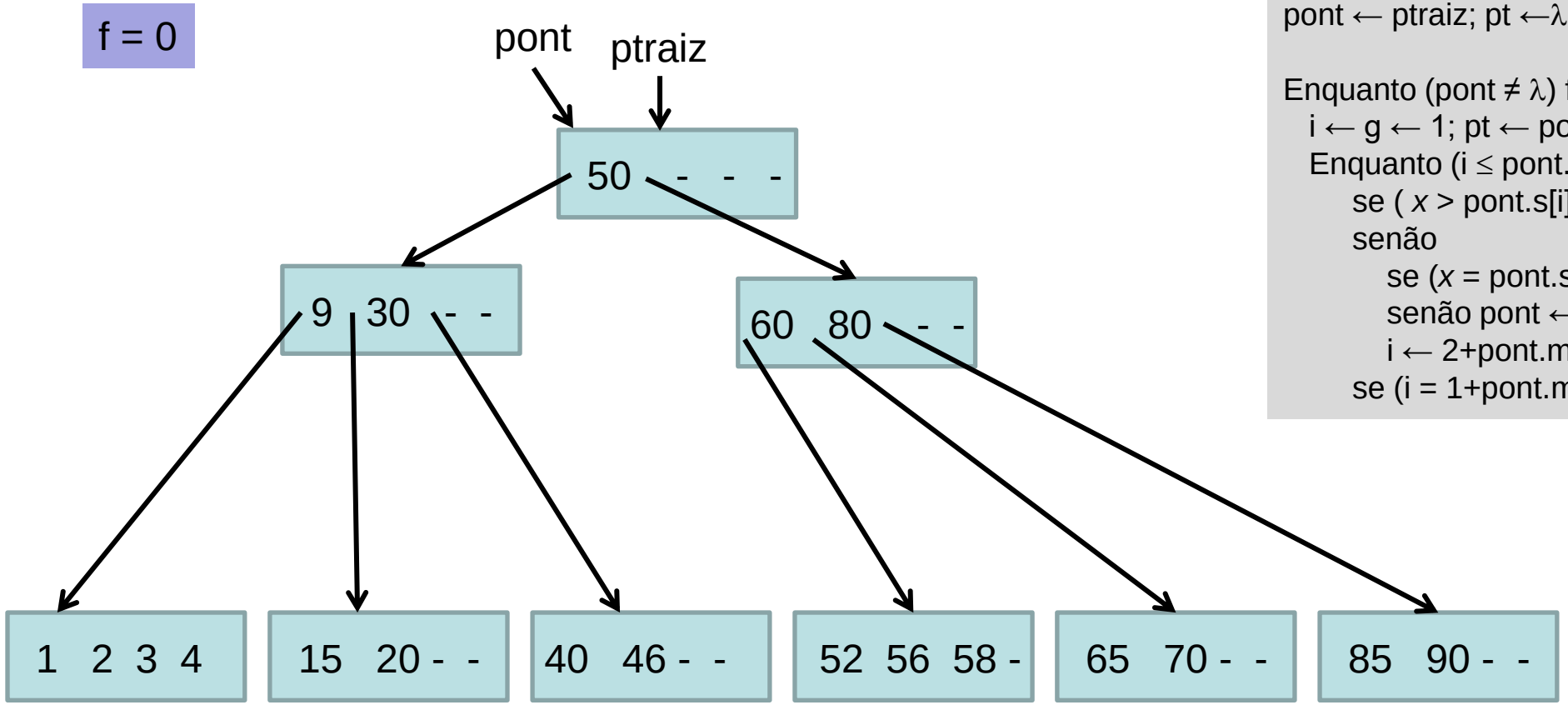
pont ← ptraiz; pt ←λ; f ← 0

Enquanto (pont ≠ λ) faça
   i ← g ← 1; pt ← pont
   Enquanto (i ≤ pont.m) faça
        se ( x > pont.s[i] ) então i ← g ← i+1
        senão
            se (x = pont.s[i]) então pont ← λ; f ← 1
            senão pont ← pont.p[i-1]
            i ← 2+pont.m
        se (i = 1+pont.m) então pont ← pont.p[pont.m]
```

# Busca em Árvore B

Busca da chave 20

f = 0




```
Procedimento buscaB (x, pt, f, g)
// f = 1, sucesso; f = 0, falha
// g – posição do nó apontado por pt

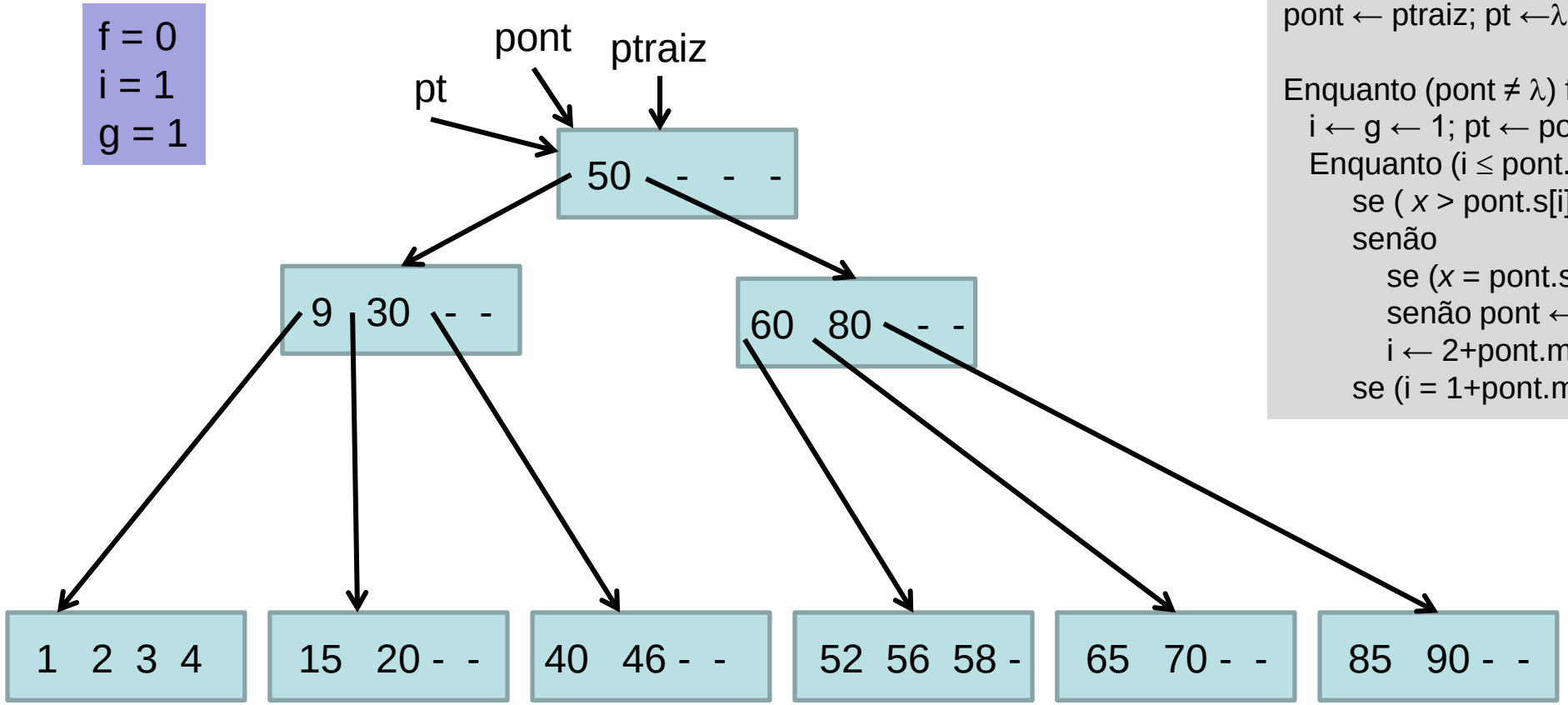
pont ← ptraiz; pt ←λ; f ← 0

Enquanto (pont ≠ λ) faça
   i ← g ← 1; pt ← pont
   Enquanto (i ≤ pont.m) faça
        se ( x > pont.s[i] ) então i ← g ← i+1
        senão
            se (x = pont.s[i]) então pont ← λ; f ← 1
            senão pont ← pont.p[i-1]
            i ← 2+pont.m
        se (i = 1+pont.m) então pont ← pont.p[pont.m]
```

# Busca em Árvore B

Busca da chave x = 20

f = 0
i = 1
g = 1



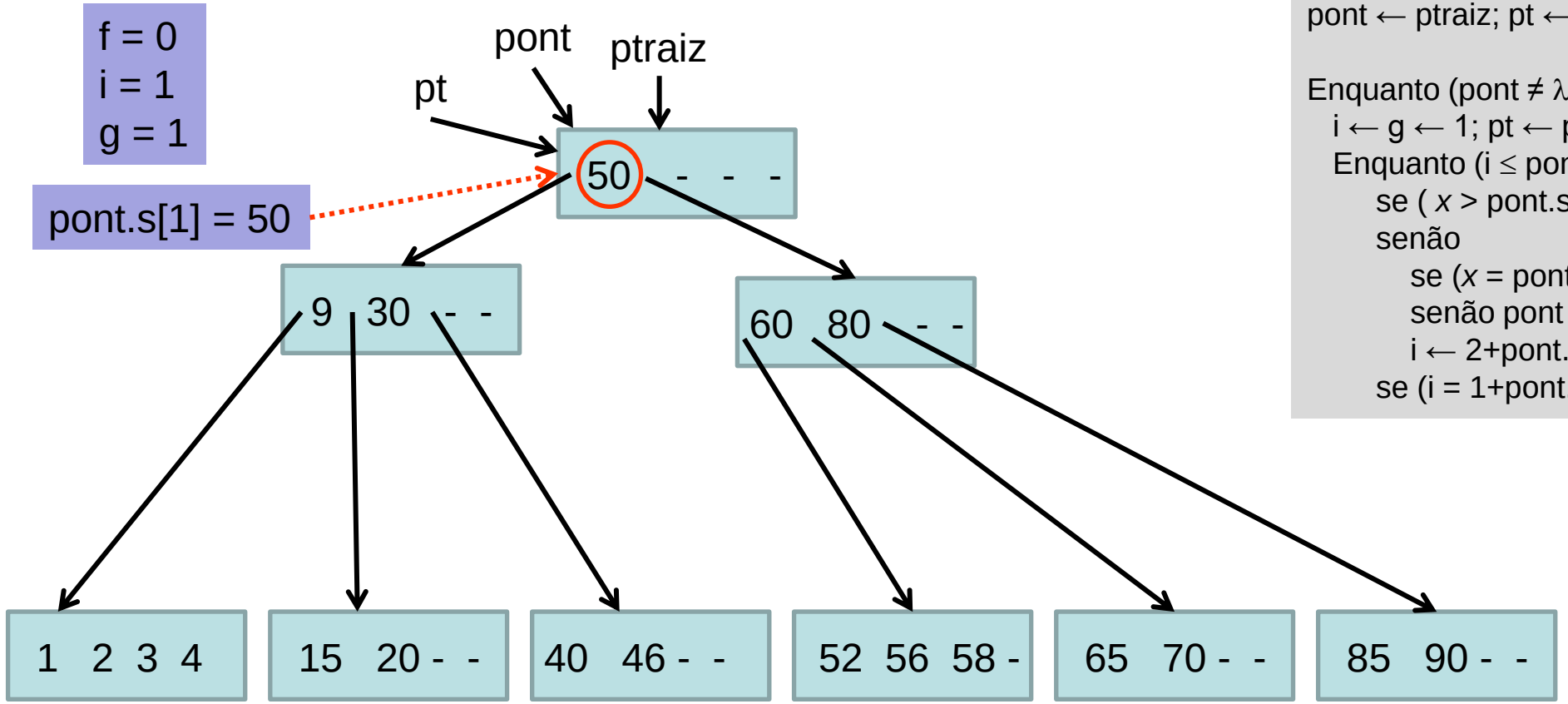

```
Procedimento buscaB (x, pt, f, g)
// f = 1, sucesso; f = 0, falha
// g – posição do nó apontado por pt

pont ← ptraiz; pt ←λ; f ← 0

Enquanto (pont ≠ λ) faça
   i ← g ← 1; pt ← pont
   Enquanto (i ≤ pont.m) faça
        se ( x > pont.s[i] ) então i ← g ← i+1
        senão
            se (x = pont.s[i]) então pont ← λ; f ← 1
            senão pont ← pont.p[i-1]
            i ← 2+pont.m
        se (i = 1+pont.m) então pont ← pont.p[pont.m]
```

# Busca em Árvore B

Busca da chave x = 20



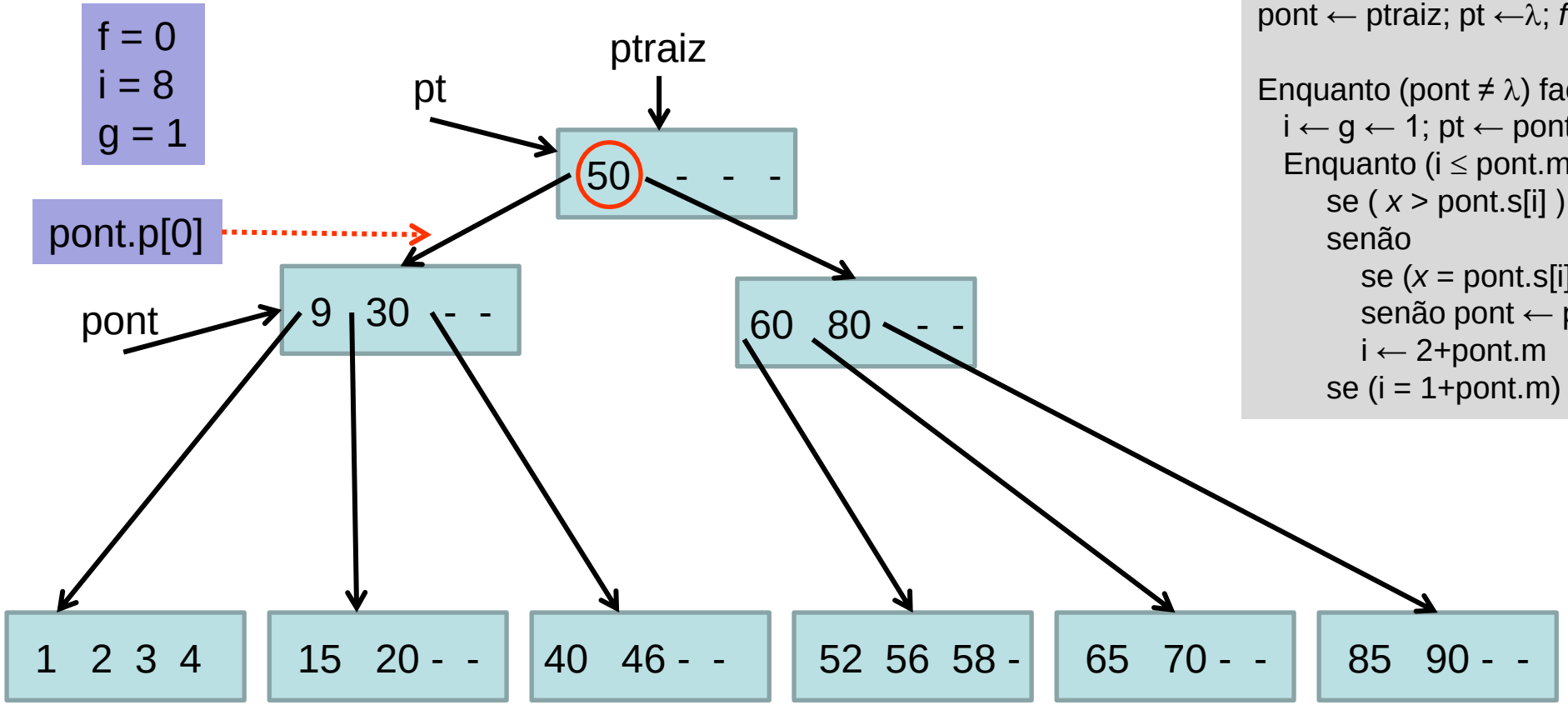

```
Procedimento buscaB (x, pt, f, g)
// f = 1, sucesso; f = 0, falha
// g – posição do nó apontado por pt

pont ← ptraiz; pt ←λ; f ← 0

Enquanto (pont ≠ λ) faça
   i ← g ← 1; pt ← pont
   Enquanto (i ≤ pont.m) faça
        se ( x > pont.s[i] ) então i ← g ← i+1
        senão
            se (x = pont.s[i]) então pont ← λ; f ← 1
            senão pont ← pont.p[i-1]
            i ← 2+pont.m
        se (i = 1+pont.m) então pont ← pont.p[pont.m]
```

# Busca em Árvore B

Busca da chave x = 20



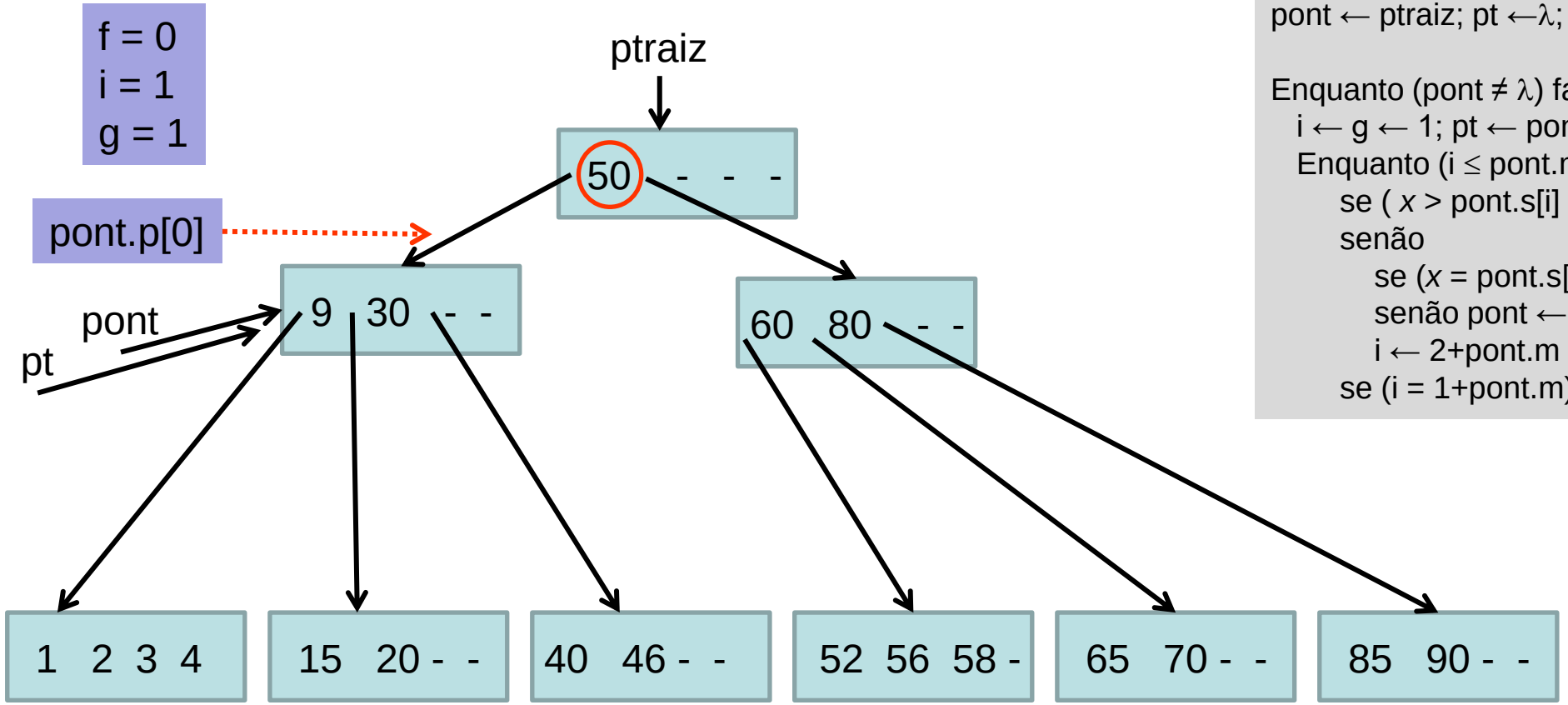

```
Procedimento buscaB (x, pt, f, g)
// f = 1, sucesso; f = 0, falha
// g – posição do nó apontado por pt

pont ← ptraiz; pt ←λ; f ← 0

Enquanto (pont ≠ λ) faça
   i ← g ← 1; pt ← pont
   Enquanto (i ≤ pont.m) faça
        se ( x > pont.s[i] ) então i ← g ← i+1
        senão
            se (x = pont.s[i]) então pont ← λ; f ← 1
            senão pont ← pont.p[i-1]
            i ← 2+pont.m
        se (i = 1+pont.m) então pont ← pont.p[pont.m]
```

# Busca em Árvore B

Busca da chave x = 20



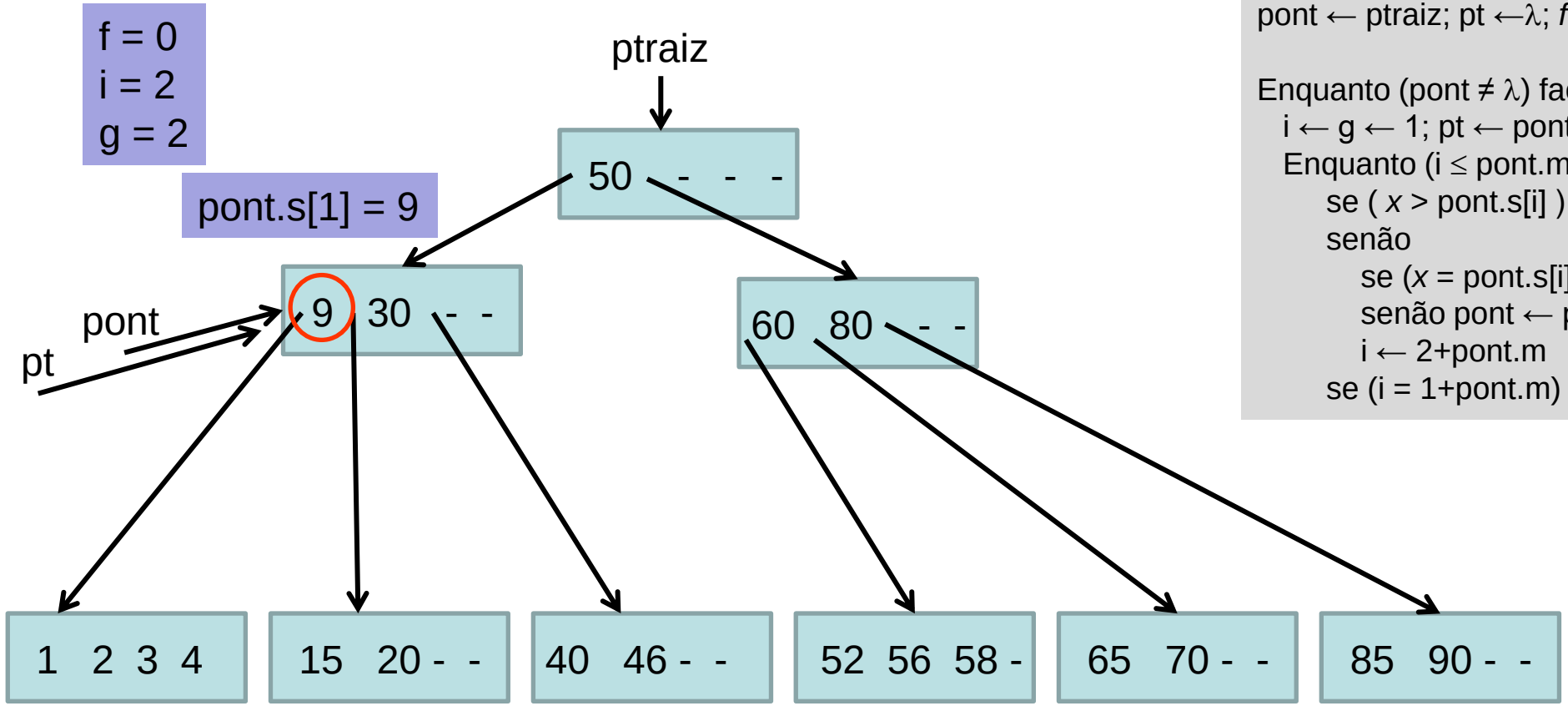

```
Procedimento buscaB (x, pt, f, g)
// f = 1, sucesso; f = 0, falha
// g – posição do nó apontado por pt

pont ← ptraiz; pt ←λ; f ← 0

Enquanto (pont ≠ λ) faça
  i ← g ← 1; pt ← pont
  Enquanto (i ≤ pont.m) faça
      se ( x > pont.s[i] ) então i ← g ← i+1
      senão
          se (x = pont.s[i]) então pont ← λ; f ← 1
          senão pont ← pont.p[i-1]
          i ← 2+pont.m
      se (i = 1+pont.m) então pont ← pont.p[pont.m]
```

# Busca em Árvore B

Busca da chave x = 20



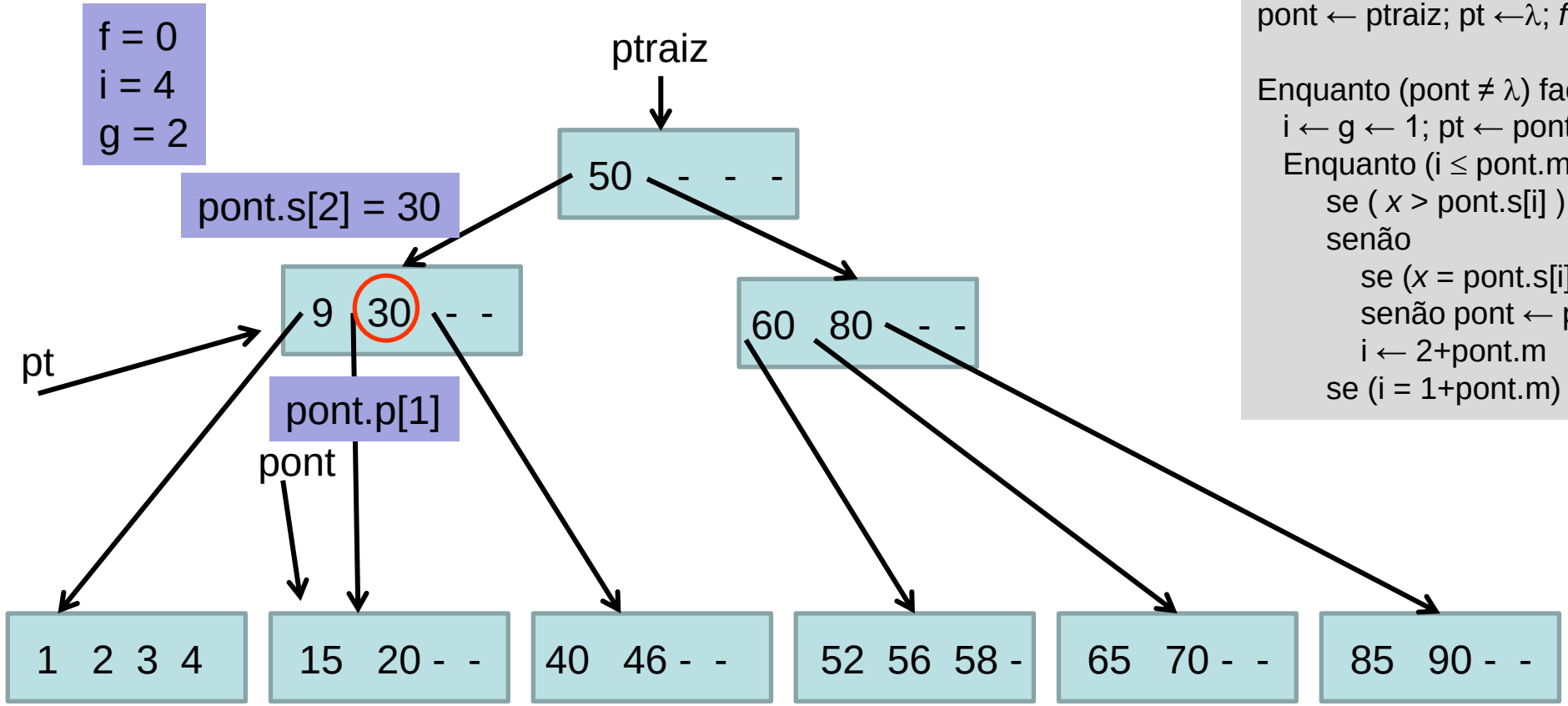

```
Procedimento buscaB (x, pt, f, g)
// f = 1, sucesso; f = 0, falha
// g – posição do nó apontado por pt

pont ← ptraiz; pt ←λ; f ← 0

Enquanto (pont ≠ λ) faça
   i ← g ← 1; pt ← pont
   Enquanto (i ≤ pont.m) faça
        se ( x > pont.s[i] ) então i ← g ← i+1
        senão
            se (x = pont.s[i]) então pont ← λ; f ← 1
            senão pont ← pont.p[i-1]
            i ← 2+pont.m
        se (i = 1+pont.m) então pont ← pont.p[pont.m]
```

# Busca em Árvore B

Busca da chave x = 20




```
Procedimento buscaB (x, pt, f, g)
// f = 1, sucesso; f = 0, falha
// g – posição do nó apontado por pt

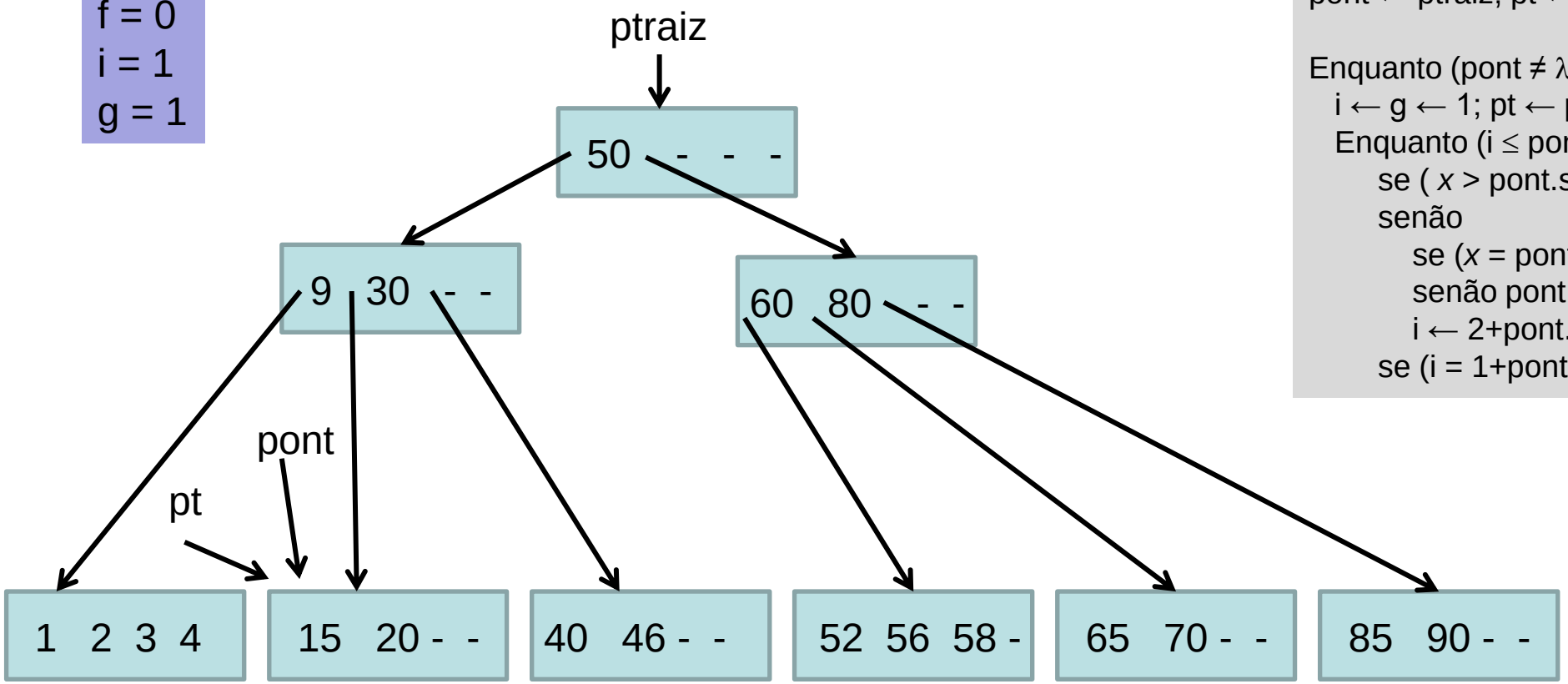
pont ← ptraiz; pt ←λ; f ← 0

Enquanto (pont ≠ λ) faça
   i ← g ← 1; pt ← pont
   Enquanto (i ≤ pont.m) faça
        se ( x > pont.s[i] ) então i ← g ← i+1
        senão
            se (x = pont.s[i]) então pont ← λ; f ← 1
            senão pont ← pont.p[i-1]
            i ← 2+pont.m
        se (i = 1+pont.m) então pont ← pont.p[pont.m]
```

# Busca em Árvore B

Busca da chave x = 20

f = 0
i = 1
g = 1




```
Procedimento buscaB (x, pt, f, g)
// f = 1, sucesso; f = 0, falha
// g – posição do nó apontado por pt

pont ← ptraiz; pt ←λ; f ← 0

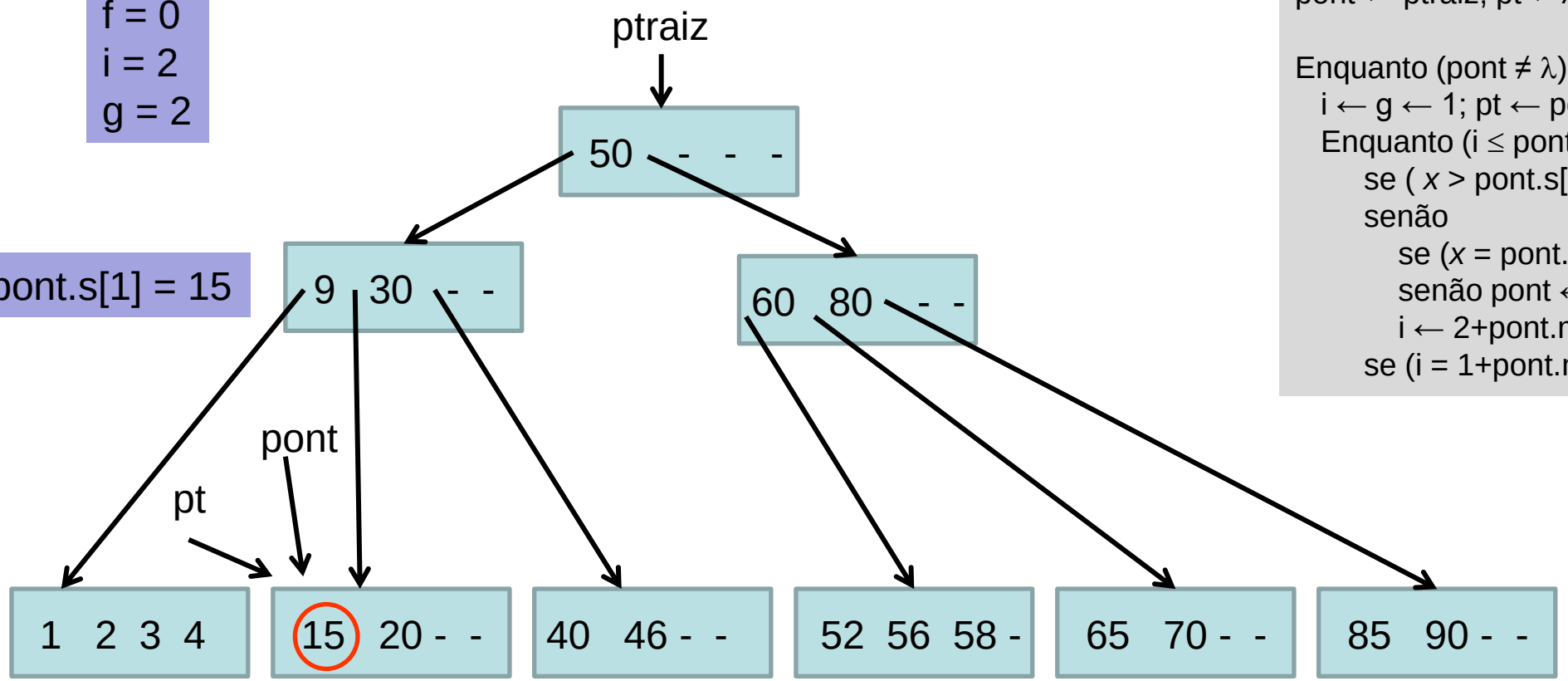
Enquanto (pont ≠ λ) faça
  i ← g ← 1; pt ← pont
  Enquanto (i ≤ pont.m) faça
      se ( x > pont.s[i] ) então i ← g ← i+1
      senão
          se (x = pont.s[i]) então pont ← λ; f ← 1
          senão pont ← pont.p[i-1]
          i ← 2+pont.m
      se (i = 1+pont.m) então pont ← pont.p[pont.m]
```

# Busca em Árvore B

Busca da chave x = 20

f = 0
i = 2
g = 2

pont.s[1] = 15




```
Procedimento buscaB (x, pt, f, g)
// f = 1, sucesso; f = 0, falha
// g – posição do nó apontado por pt

pont ← ptraiz; pt ←λ; f ← 0

Enquanto (pont ≠ λ) faça
   i ← g ← 1; pt ← pont
   Enquanto (i ≤ pont.m) faça
        se ( x > pont.s[i] ) então i ← g ← i+1
        senão
            se (x = pont.s[i]) então pont ← λ; f ← 1
            senão pont ← pont.p[i-1]
            i ← 2+pont.m
        se (i = 1+pont.m) então pont ← pont.p[pont.m]
```

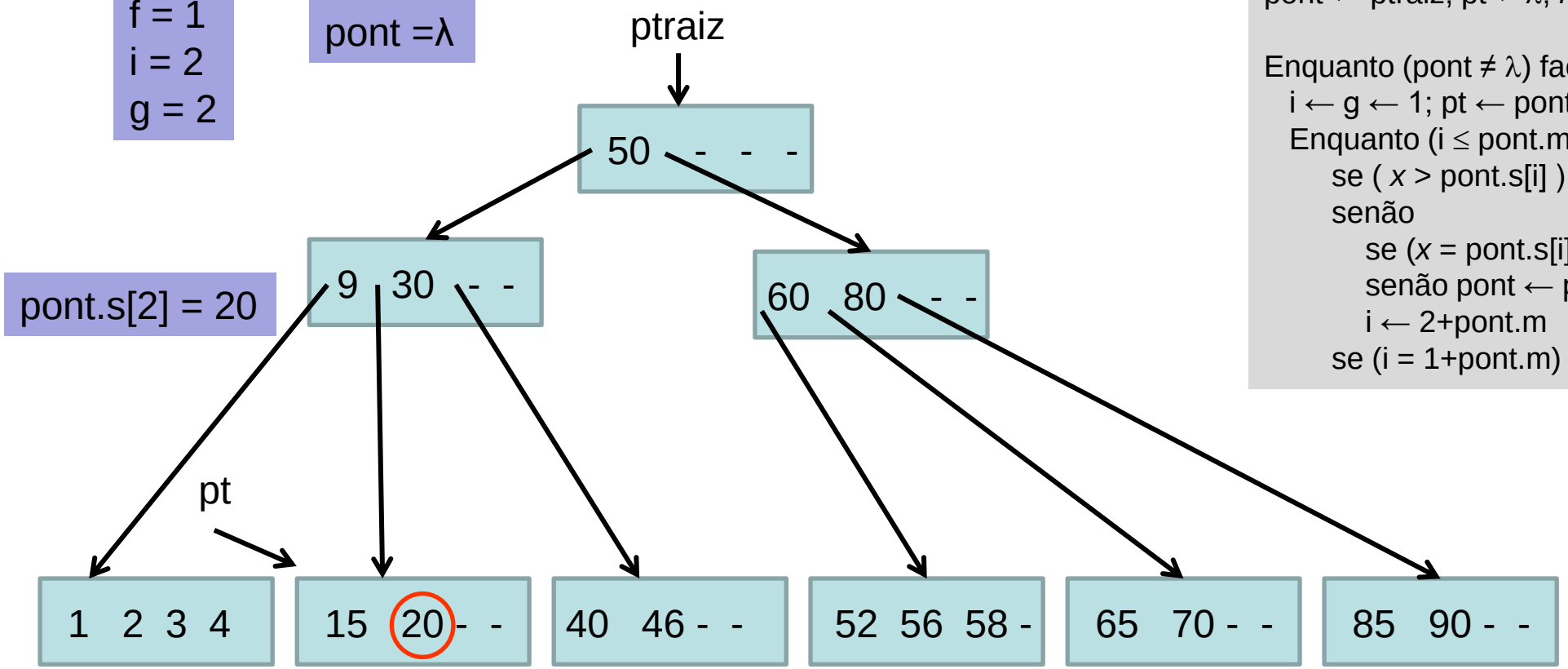
# Busca em Árvore B

Busca da chave x = 20

f = 1
i = 2
g = 2

pont =λ

pont.s[2] = 20



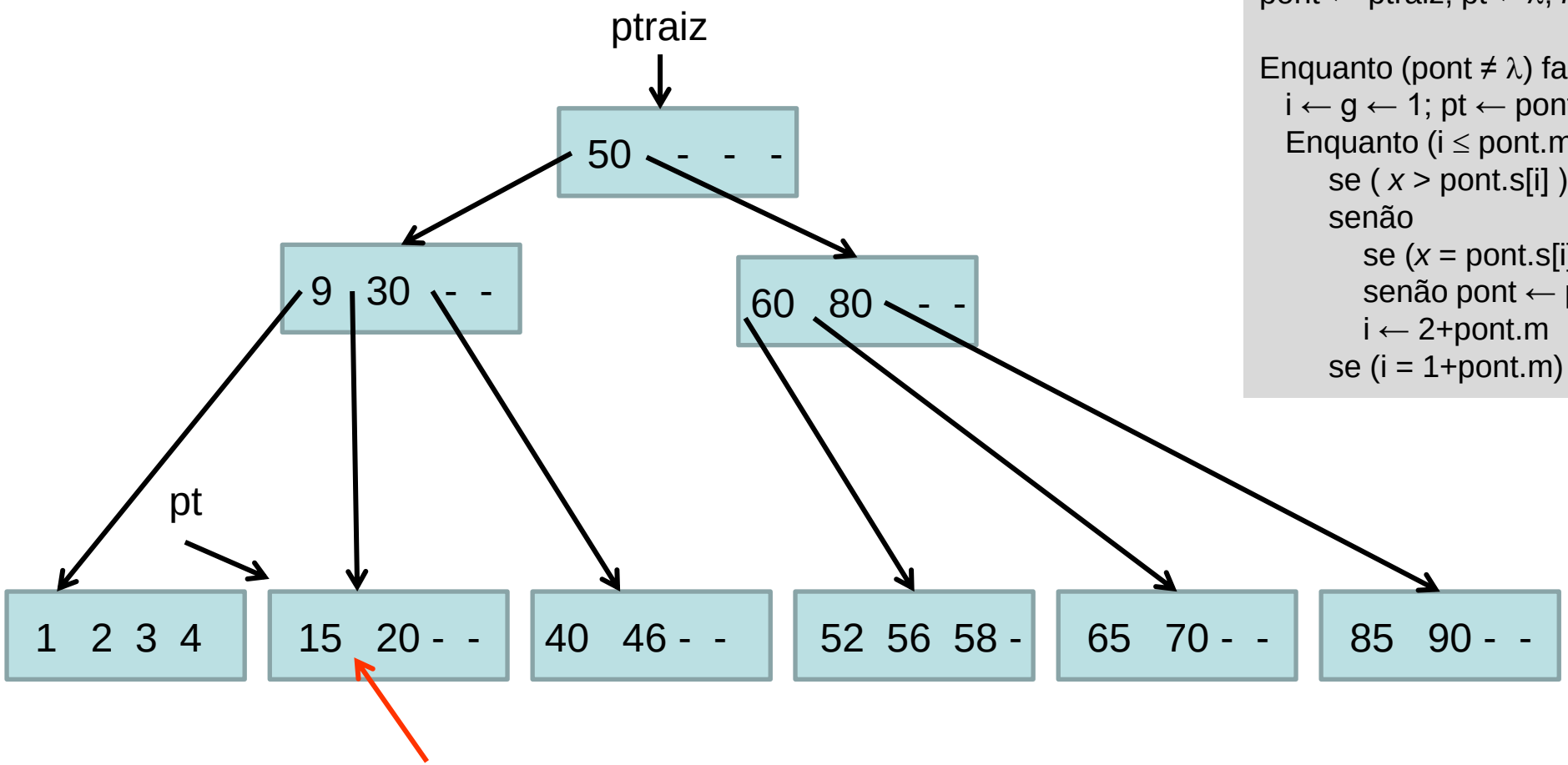

```
Procedimento buscaB (x, pt, f, g)
// f = 1, sucesso; f = 0, falha
// g – posição do nó apontado por pt

pont ← ptraiz; pt ←λ; f ← 0

Enquanto (pont ≠ λ) faça
   i ← g ← 1; pt ← pont
   Enquanto (i ≤ pont.m) faça
        se ( x > pont.s[i] ) então i ← g ← i+1
        senão
            se (x = pont.s[i]) então pont ← λ; f ← 1
            senão pont ← pont.p[i-1]
            i ← 2+pont.m
        se (i = 1+pont.m) então pont ← pont.p[pont.m]
```

# Busca em Árvore B

## Busca da chave x = 17




```
Procedimento buscaB (x, pt, f, g)
// f = 1, sucesso; f = 0, falha
// g – posição do nó apontado por pt

pont ← ptraiz; pt ←λ; f ← 0

Enquanto (pont ≠ λ) faça
   i ← g ← 1; pt ← pont
   Enquanto (i ≤ pont.m) faça
        se ( x > pont.s[i] ) então i ← g ← i+1
        senão
            se (x = pont.s[i]) então pont ← λ; f ← 1
            senão pont ← pont.p[i-1]
            i ← 2+pont.m
        se (i = 1+pont.m) então pont ← pont.p[pont.m]
```

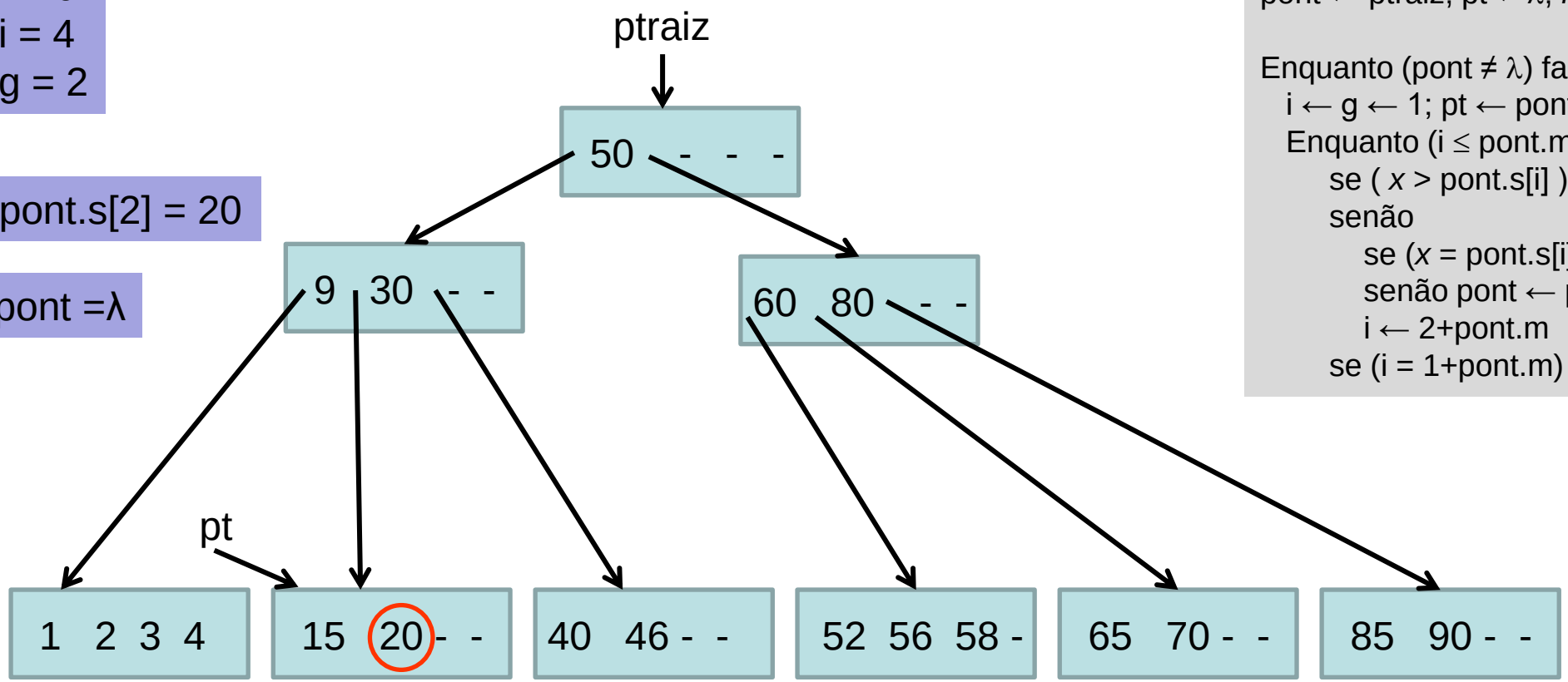
# Busca em Árvore B

## Busca da chave x = 17

f = 0
i = 4
g = 2

pont.s[2] = 20

pont =λ




```
Procedimento buscaB (x, pt, f, g)
// f = 1, sucesso; f = 0, falha
// g – posição do nó apontado por pt

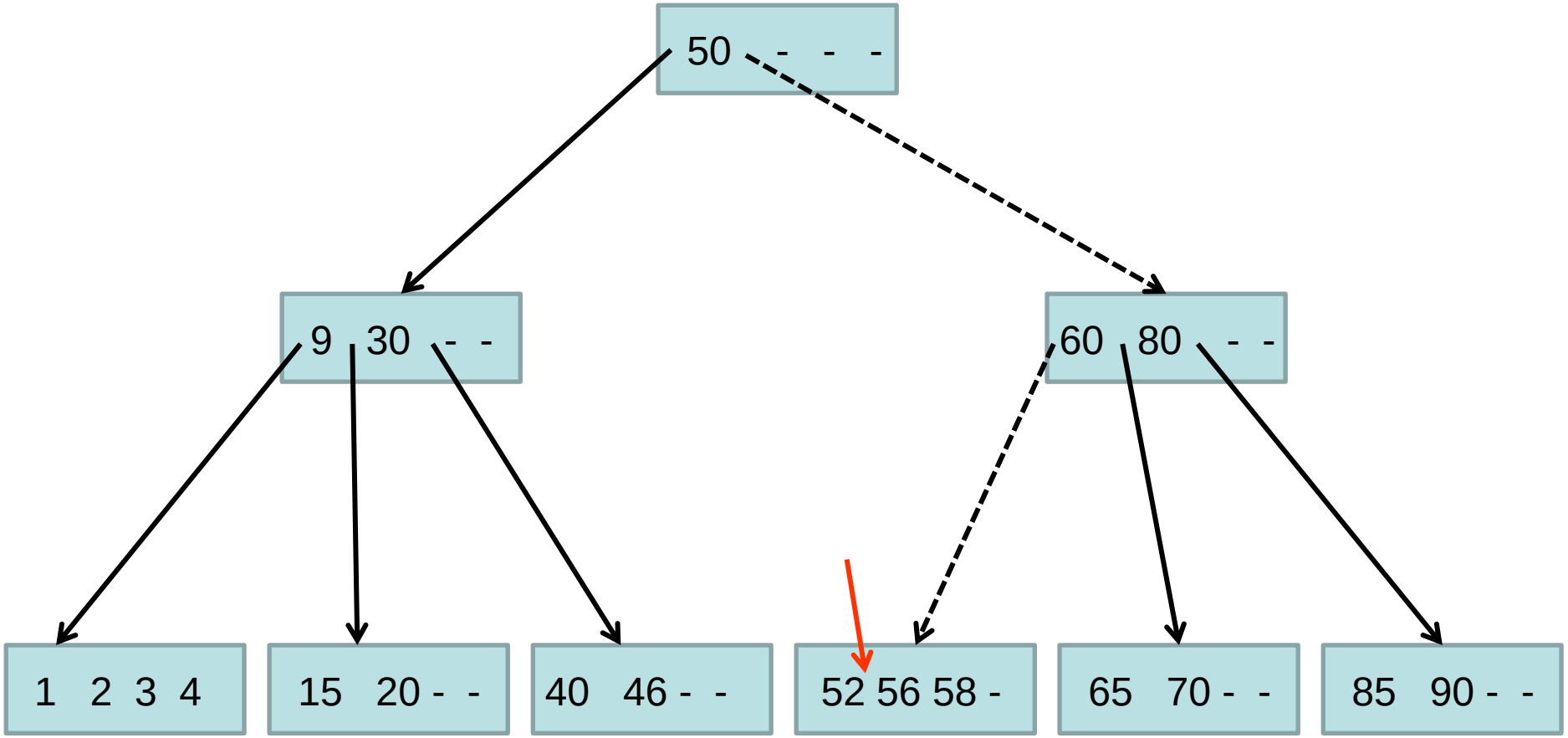
pont ← ptraiz; pt ←λ; f ← 0

Enquanto (pont ≠ λ) faça
   i ← g ← 1; pt ← pont
   Enquanto (i ≤ pont.m) faça
        se ( x > pont.s[i] ) então i ← g ← i+1
        senão
            se (x = pont.s[i]) então pont ← λ; f ← 1
            senão pont ← pont.p[i-1]
            i ← 2+pont.m
        se (i = 1+pont.m) então pont ← pont.p[pont.m]
```

# Inserção e Remoção em Árvore B

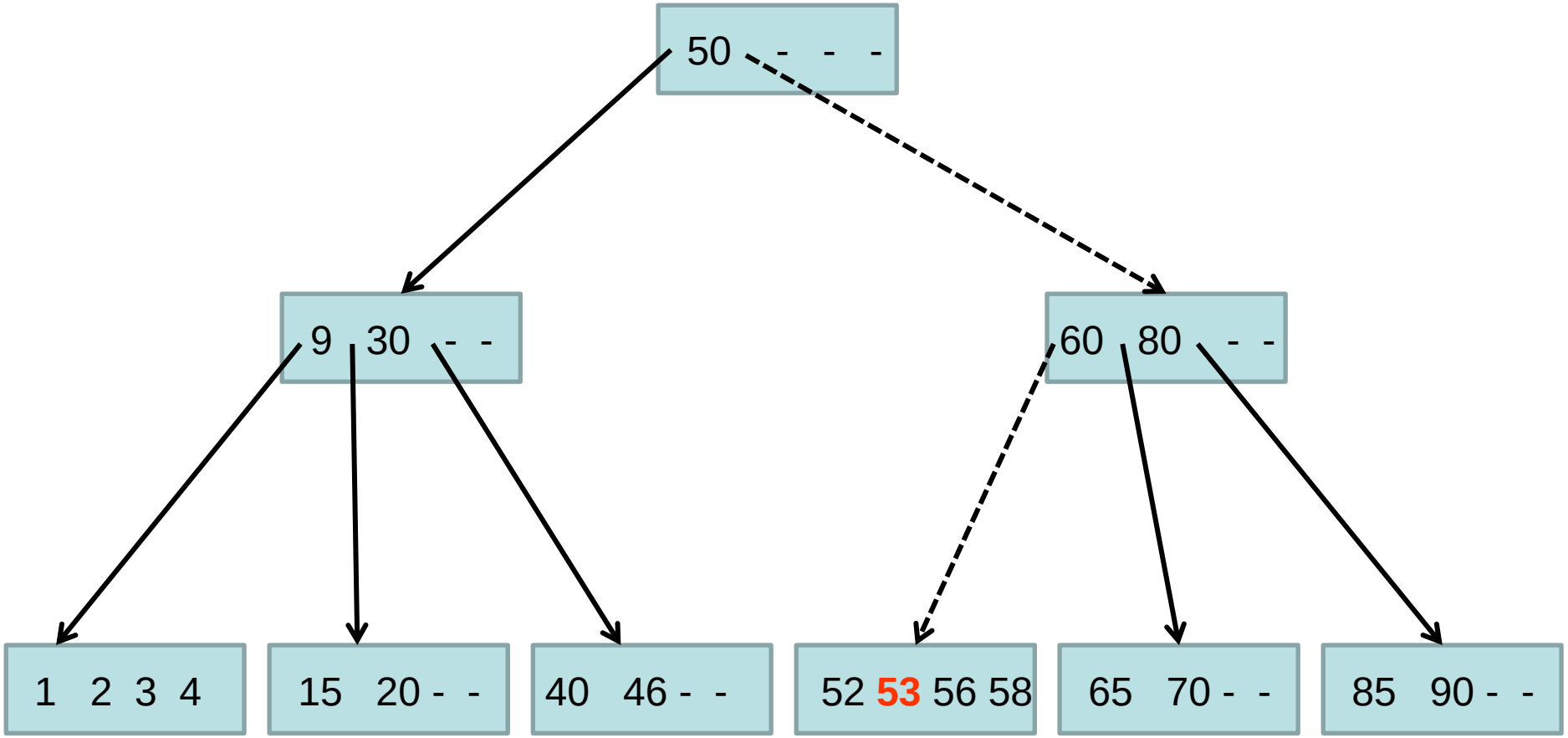
# Inserção em Árvore B

Inserção da chave 53



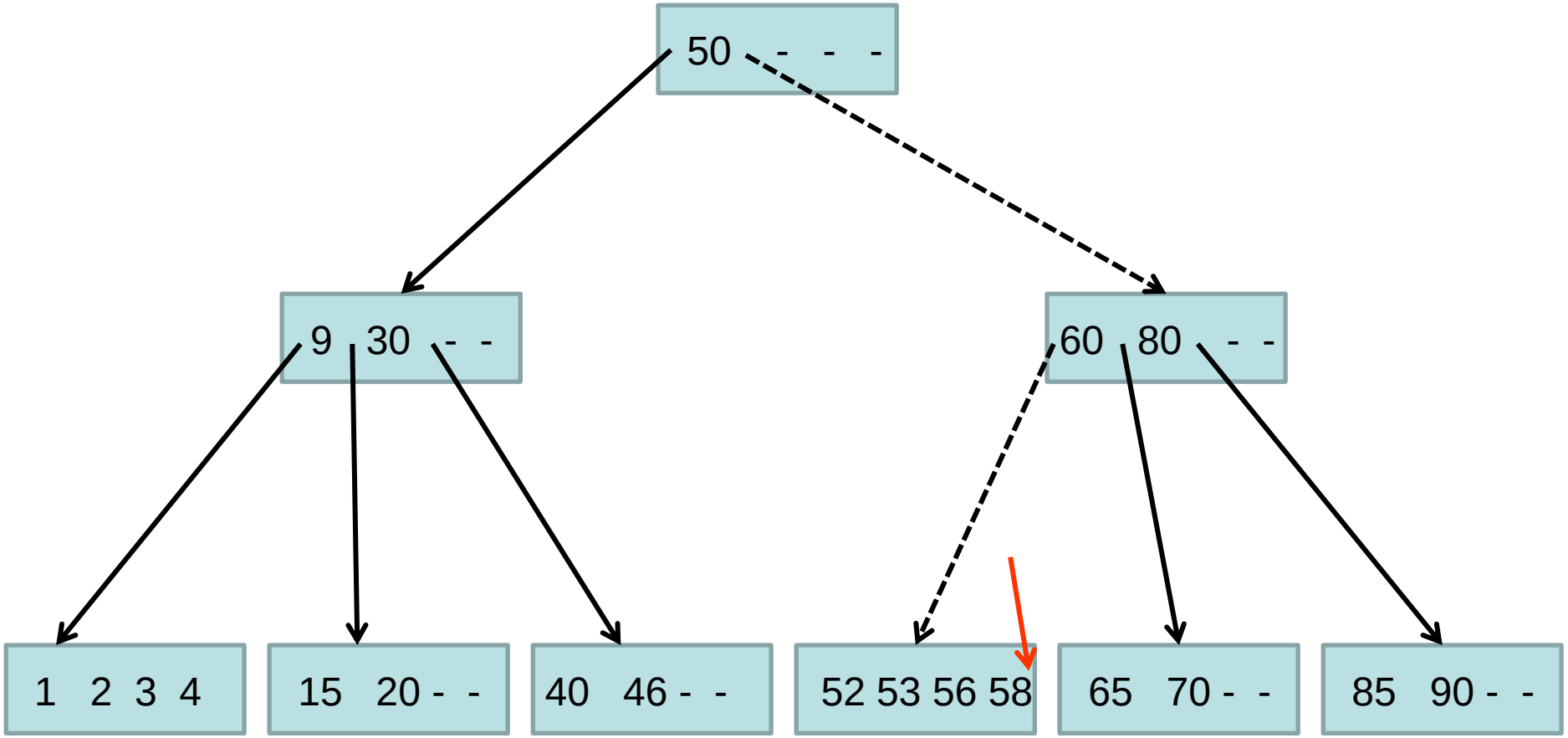
# Inserção em Árvore B

Inserção da chave 53



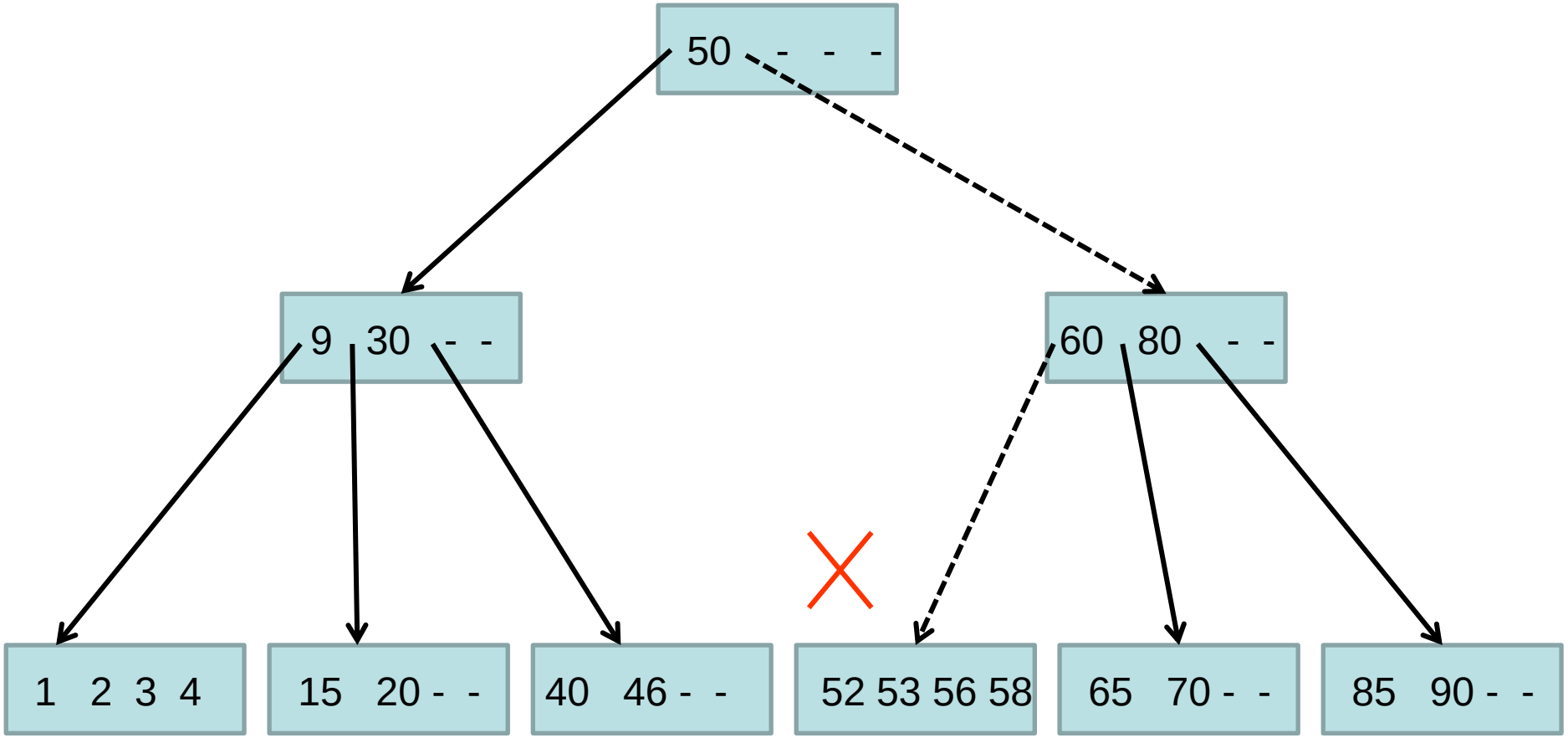
# Inserção em Árvore B

Inserção da chave 59



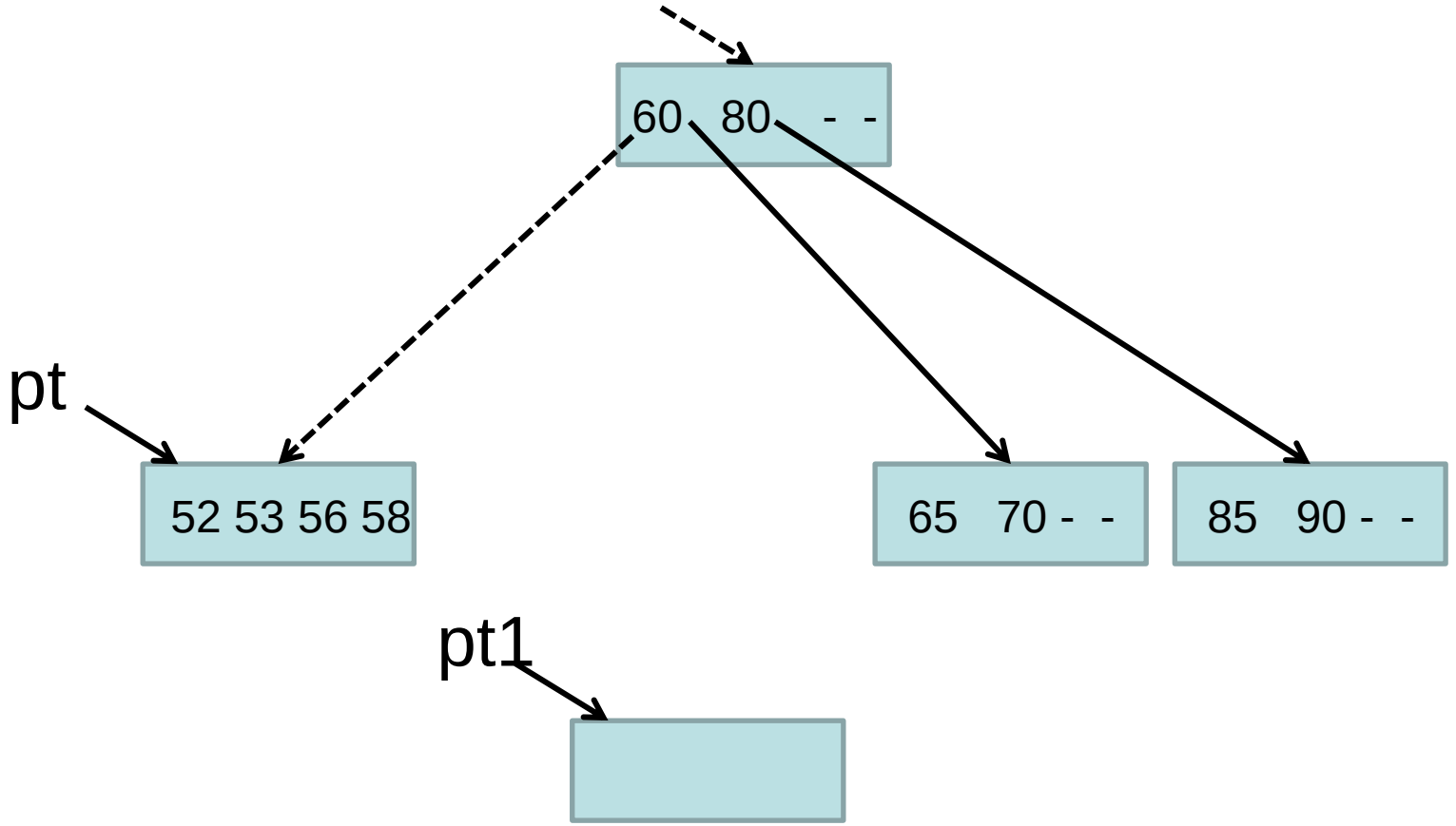
# Inserção em Árvore B

Inserção da chave 59



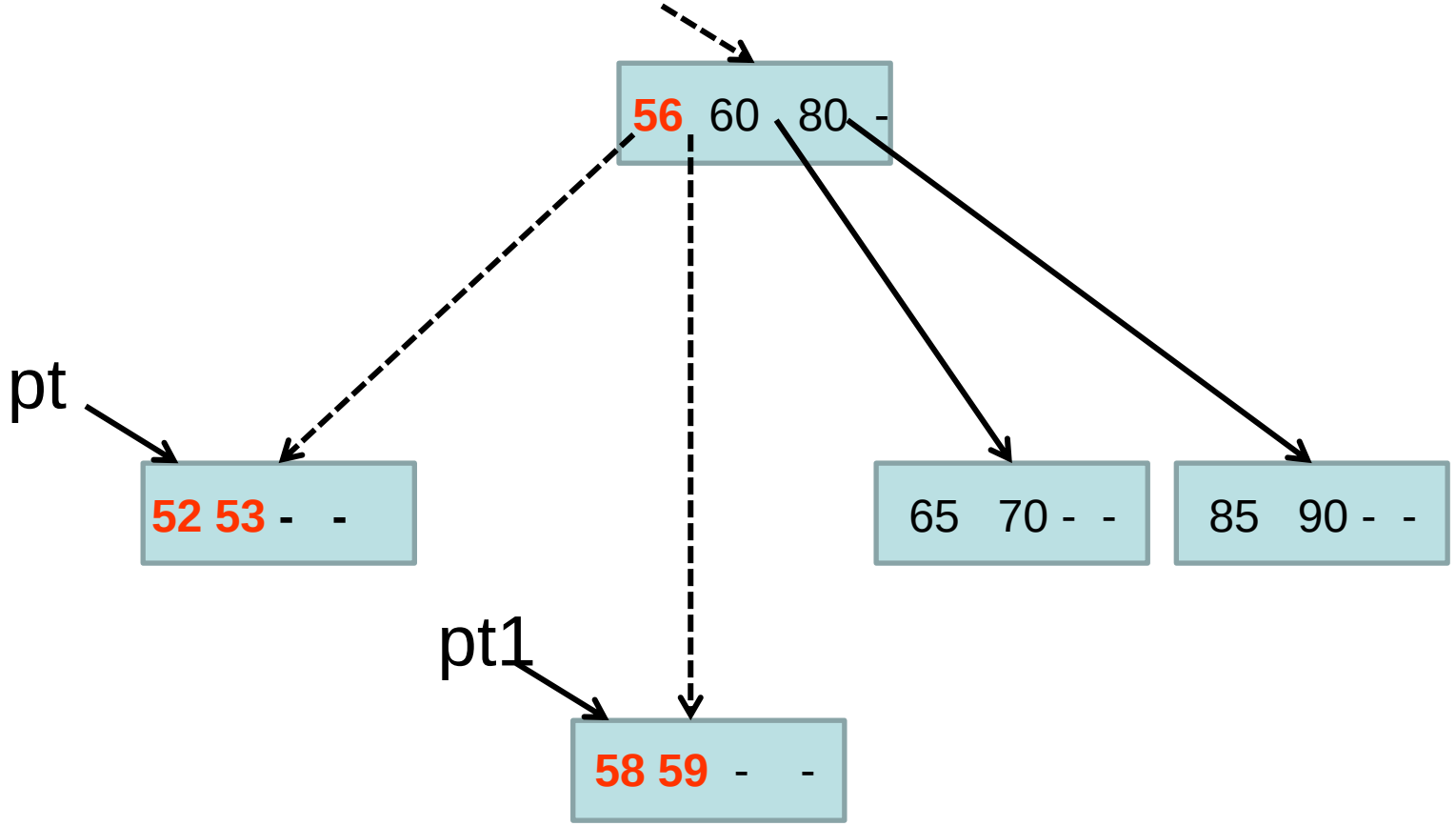
# Inserção em Árvore B

## Inserção da chave 59 – SOLUÇÃO: Cisão de Página



# Inserção em Árvore B

## Inserção da chave 59 – SOLUÇÃO: Cisão de Página



# Inserção em Árvore B

Caso em que a página fica com 2d + 1 chaves

*Solução*: **Cisão de Páginas**

Considere P a página com excesso de chaves.
P será transformada em 2 páginas.

Considere pt o ponteiro que aponta para P.

Permanecerão **d chaves** em P. Uma nova página Q (apontada por pt1) é solicitada. Q recebe **d chaves**.

# Inserção em Árvore B

A chave que sobra, da posição central de P, $s_d + 1$, forma com o ponteiro pt1, uma nova entrada na página W pai de P.

Final:
P – $p_0$, ($s_1$,$p_1$), …, ($s_d$,$p_d$)

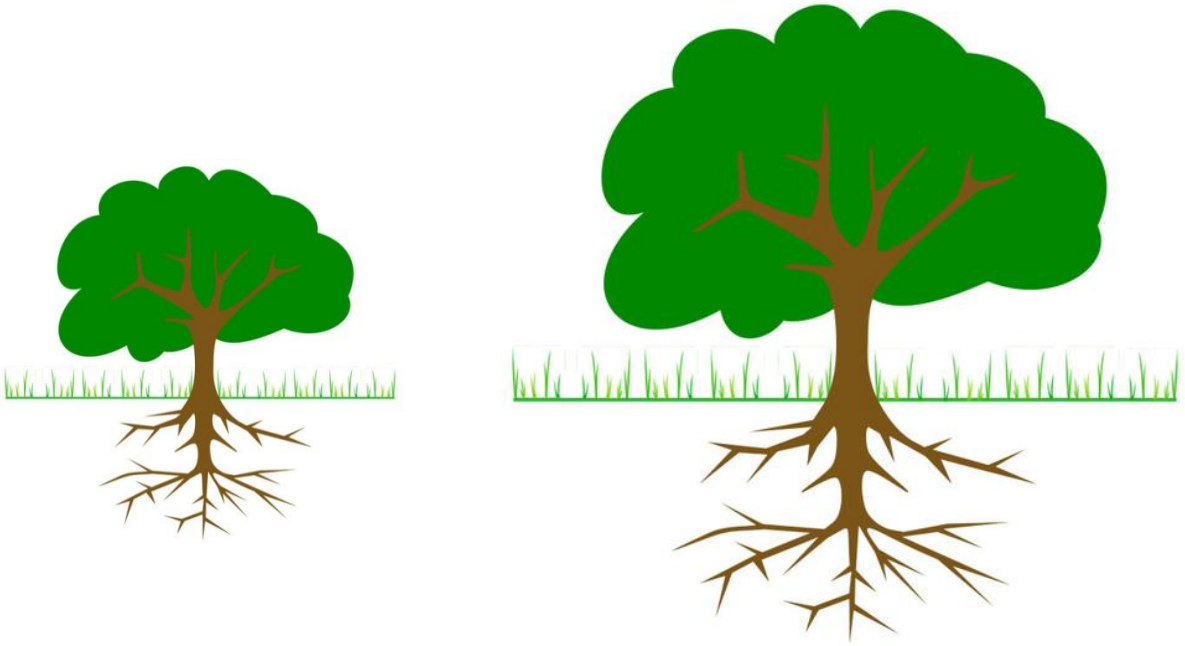
Q – $p_{d+1}$, ($s_{d+2}$,$p_{d+2}$), …, ($s_{2d+1}$,$p_{2d+1}$)

W – recebe ($s_{d+1}$,$p_{d+1}$)

OBS. Pode ser necessário fazer a cisão de W.

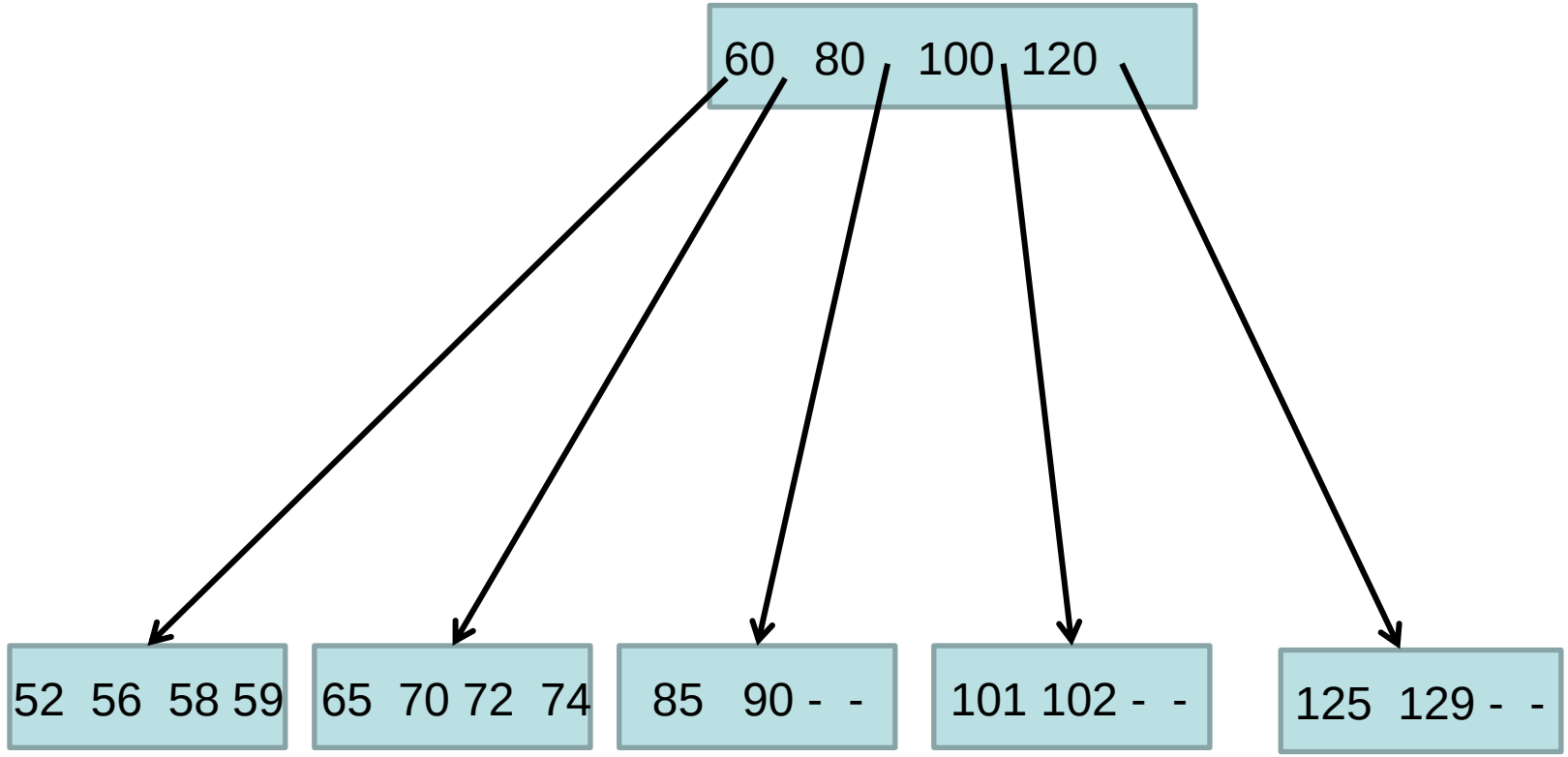
# Inserção em Árvore B

A cisão é propagável e pode chegar a raiz da árvore, podendo acarretar o aumento da altura da árvore, que se mantem, entretanto, com as folhas no mesmo nível, uma vez que surge uma nova raiz.



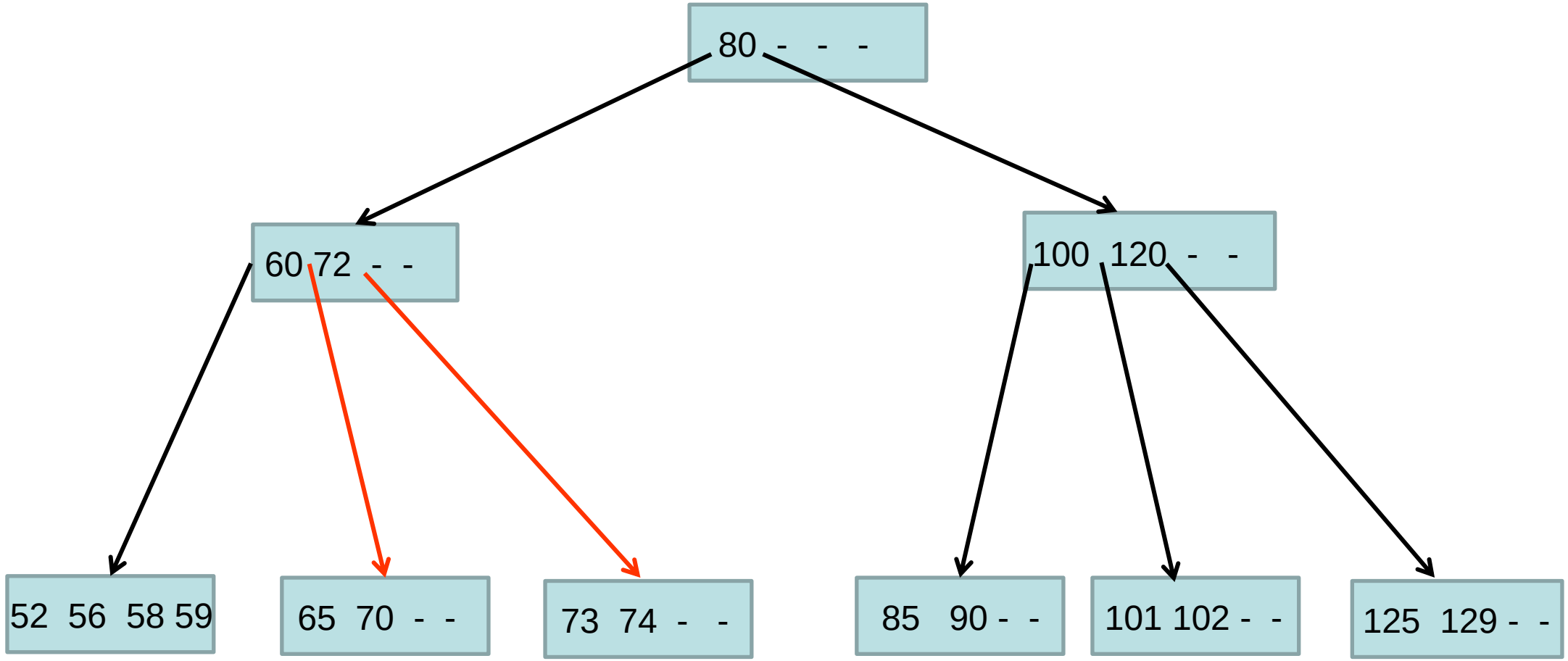
# Exercício

1. Considerando a árvore B de ordem 2 abaixo, inserir a chave 73.



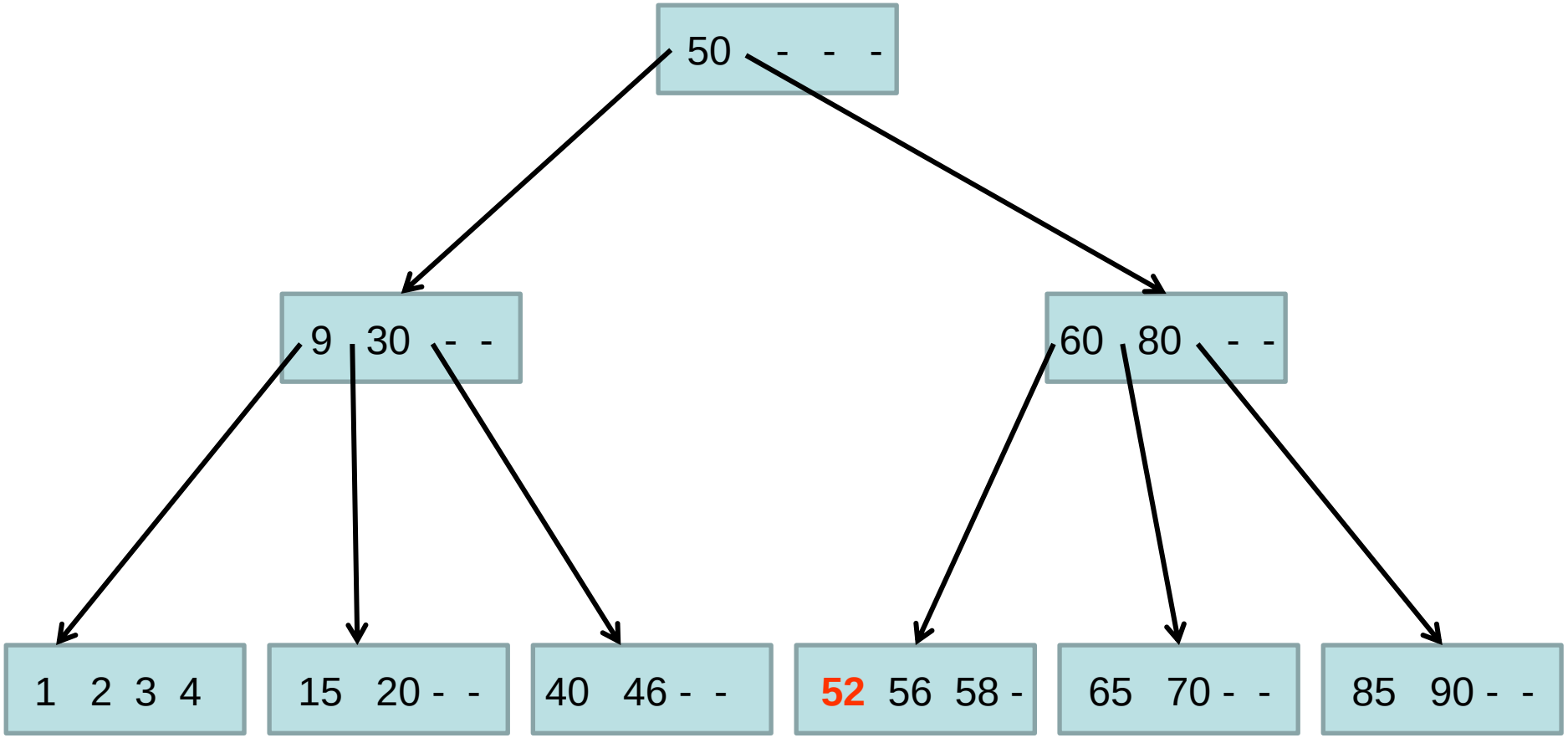
# Exercício

1. chave 73 inserida



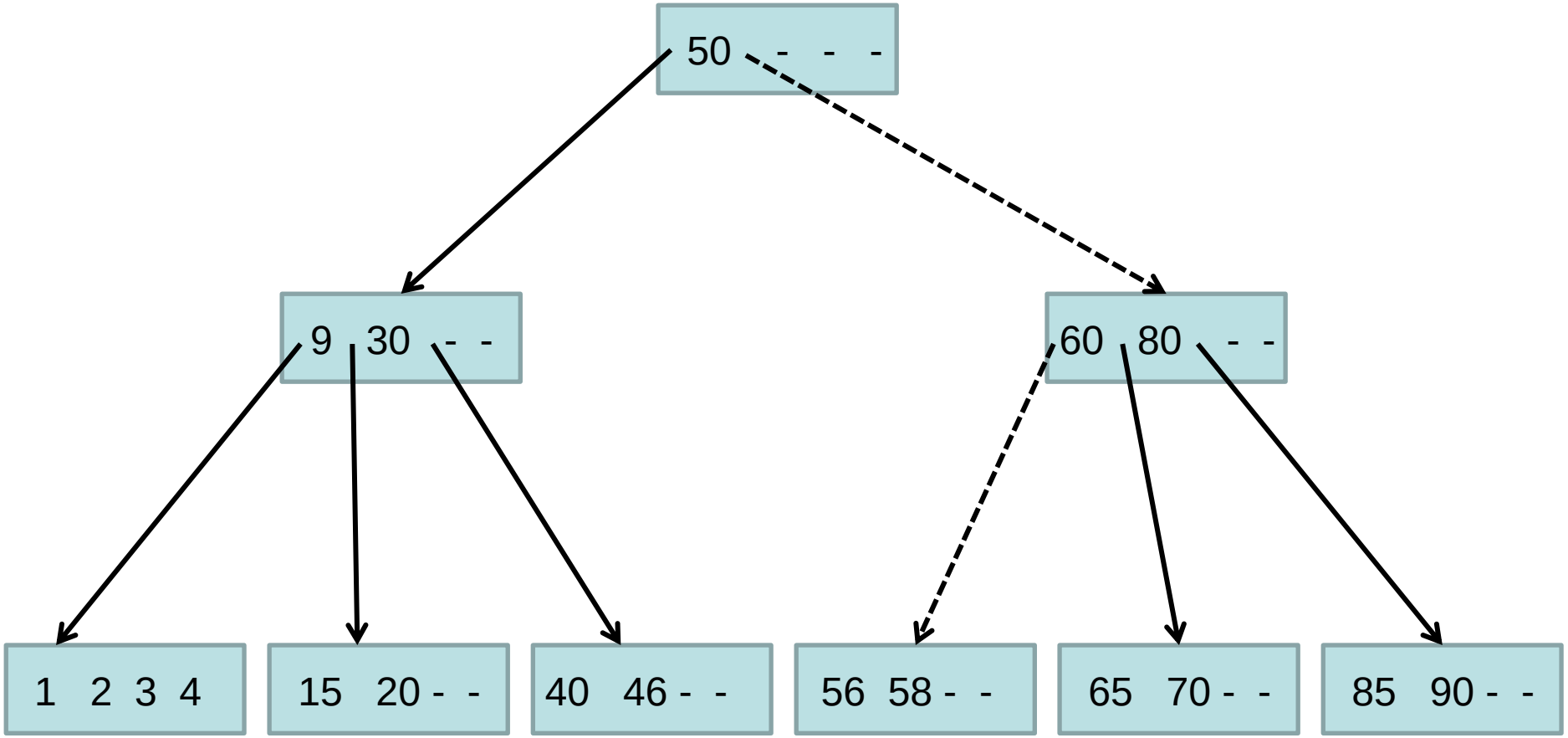
# Remoção em Árvore B

Exemplo: Remoção da chave 52 (chave na folha)



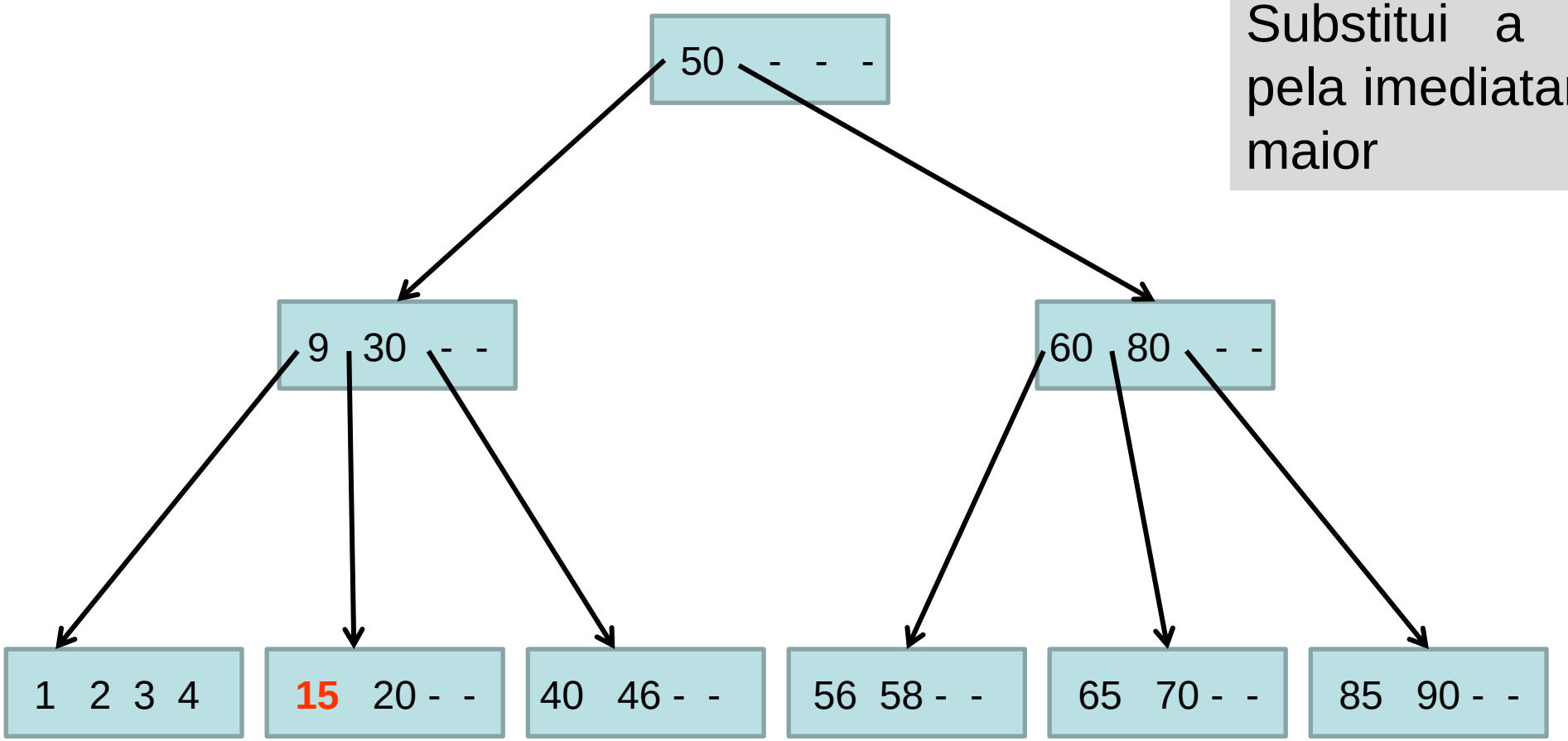
# Remoção em Árvore B

Exemplo: Remoção da chave 52 (chave na folha)



# Remoção em Árvore B

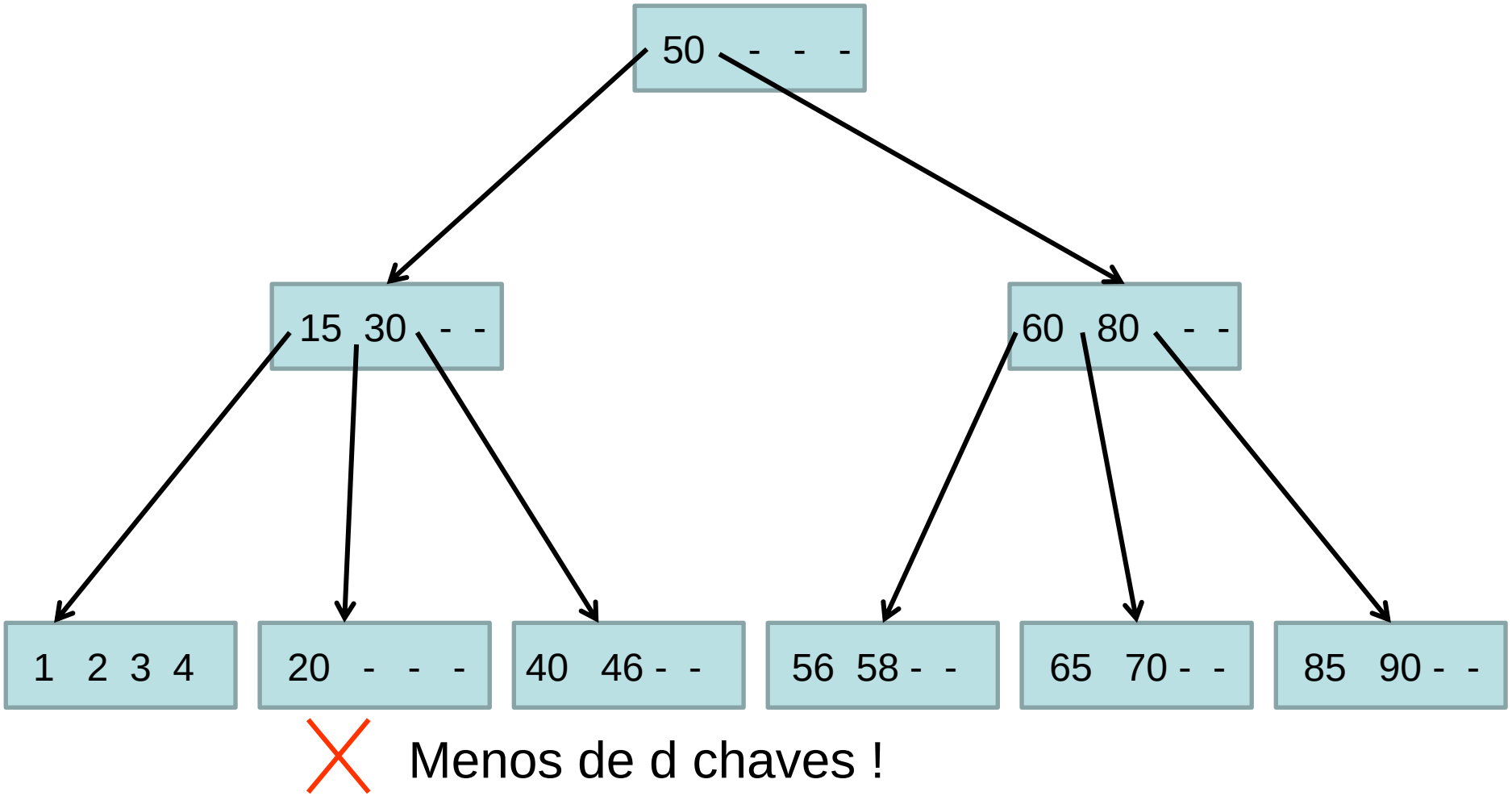
Exemplo: Remoção da chave 9 (nó não folha)




Substitui a chave pela imediatamente maior

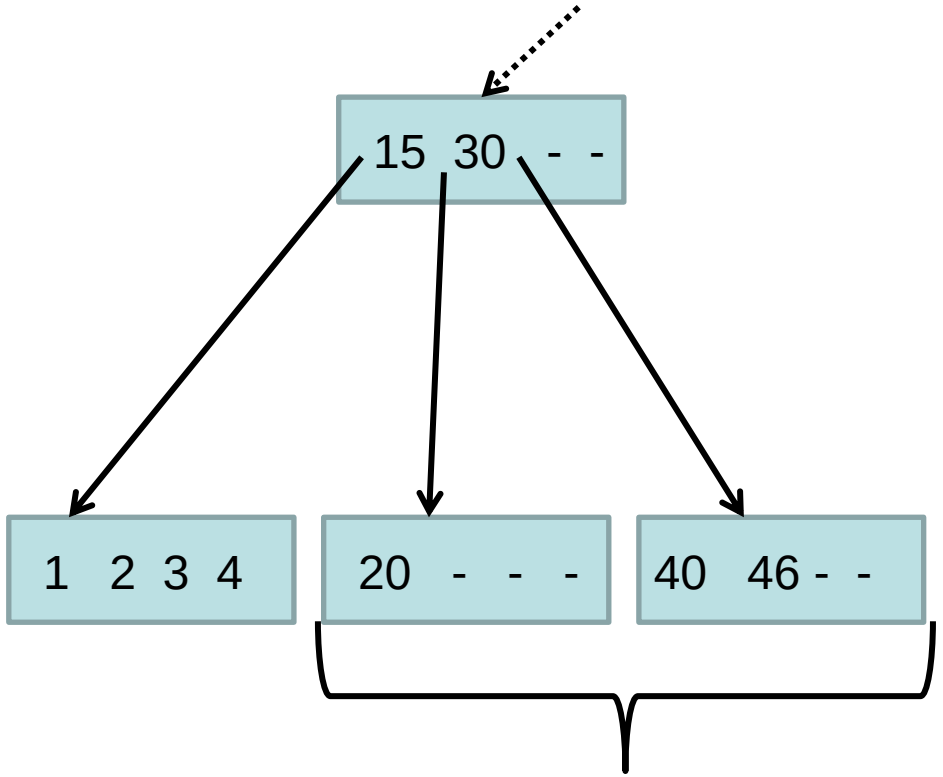
# Remoção em Árvore B

Exemplo: Remoção da chave 9 (nó não folha)



# Remoção em Árvore B

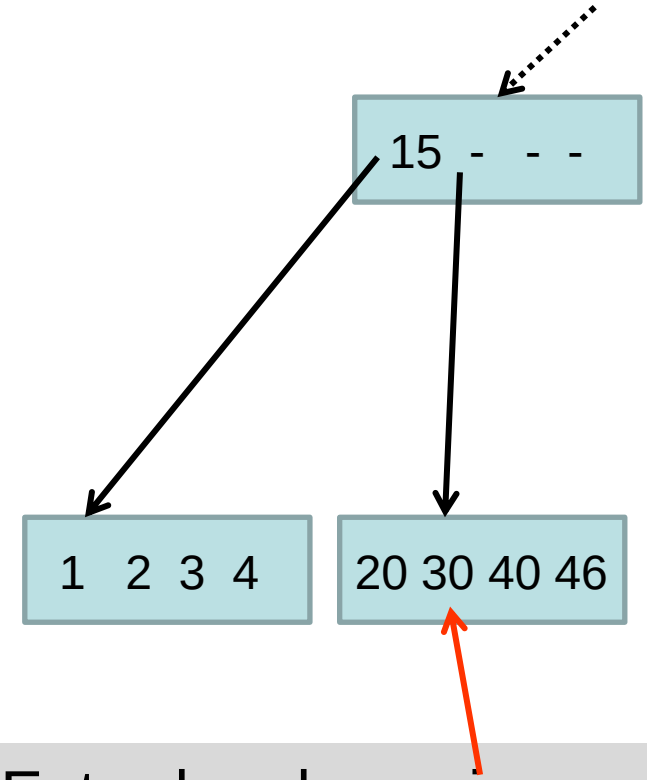
Solução: CONCATENAÇÃO




Páginas irmãs que possuem juntas menos de 2d chaves

# Remoção em Árvore B

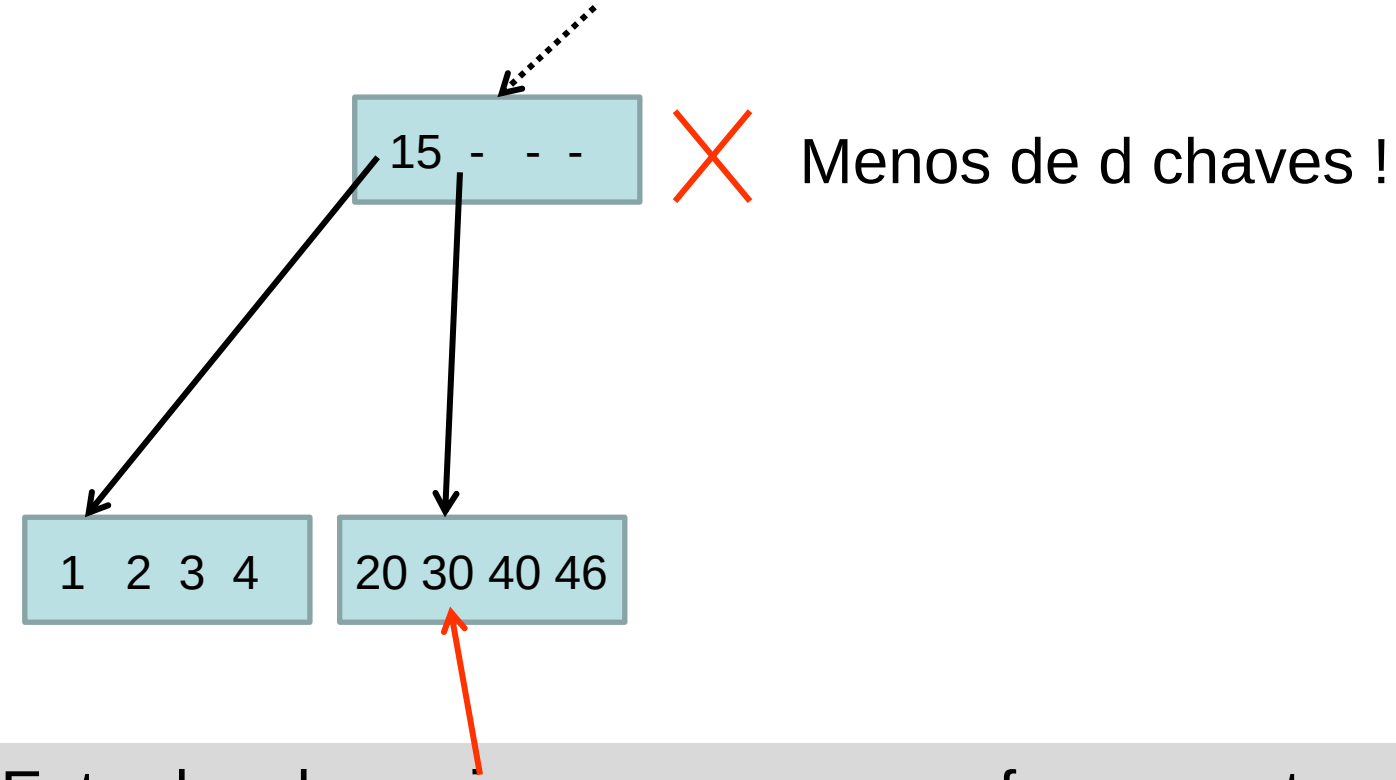
Solução: CONCATENAÇÃO




Entrada do pai que passa a fazer parte da chave resultante da concatenação

# Remoção em Árvore B

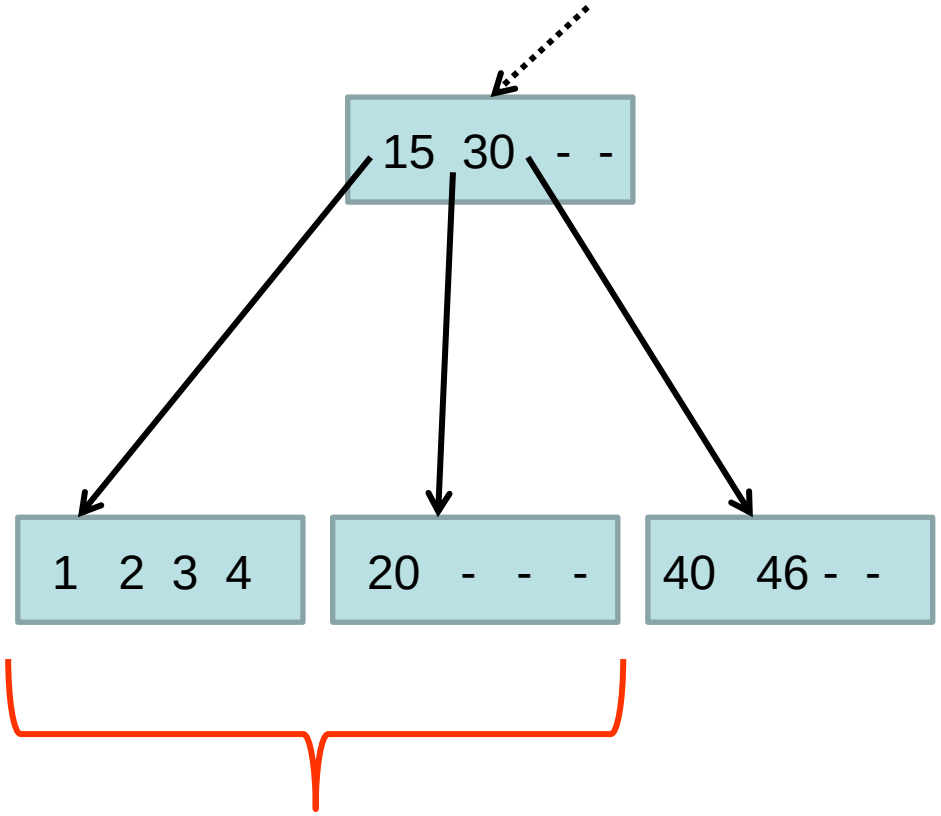
Solução: CONCATENAÇÃO




Entrada do pai que passa a fazer parte da chave resultante da concatenação

# Remoção em Árvore B

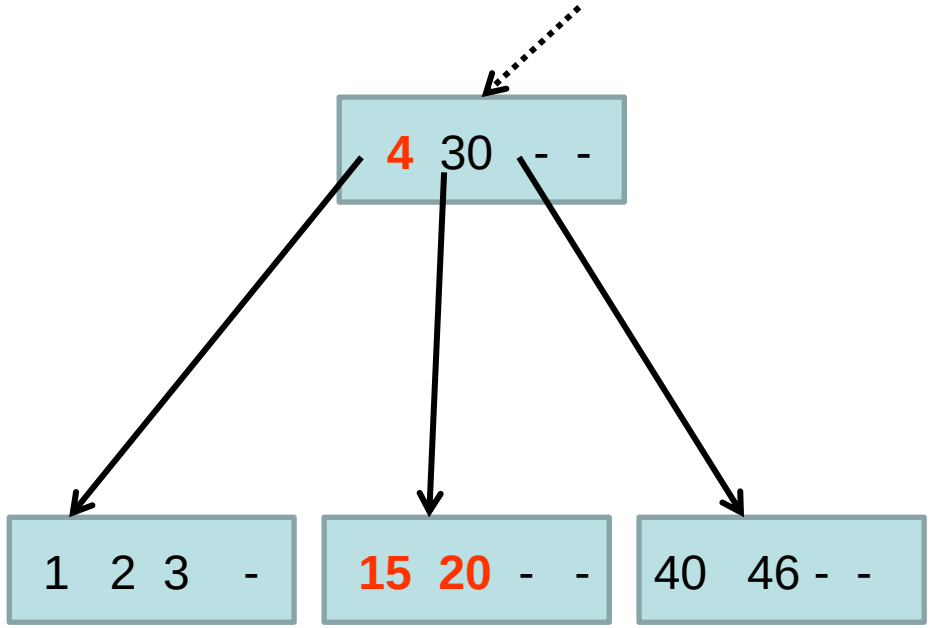
Solução: REDISTRIBUIÇÃO




páginas irmãs que possuem juntas mais de 2d chaves

# Remoção em Árvore B

Solução: REDISTRIBUIÇÃO = Concatenação + Cisão



# Remoção em Árvore B

1. A chave se encontra em uma folha

   **A entrada é retirada**

2. A chave se encontra em um nó não folha

   **A chave é substituída pela chave *y* imediatamente maior (*y está em uma folha*)**

Problema: Se ficar menos de d chaves na página !

Soluções: Concatenação

Redistribuição

# Remoção em Árvore B

## Concatenação

P e Q são irmãs adjacentes se possuem o mesmo pai W e são apontadas por ponteiros consecutivos.

Se P e Q juntas possuírem menos de 2d chaves, elas podem ser concatenadas.

P e Q são agrupadas em uma página. Para isso a entrada que se encontra no pai W entre P e Q, passa também a fazer parte da nova página.

A nova página possui 2d entradas.

# Remoção em Árvore B

## Concatenação

A concatenação é propagável e pode alterar a altura da árvore.

# Remoção em Árvore B

## Redistribuição

P e Q são irmãs adjacentes e possuem juntas 2d ou mais chaves.

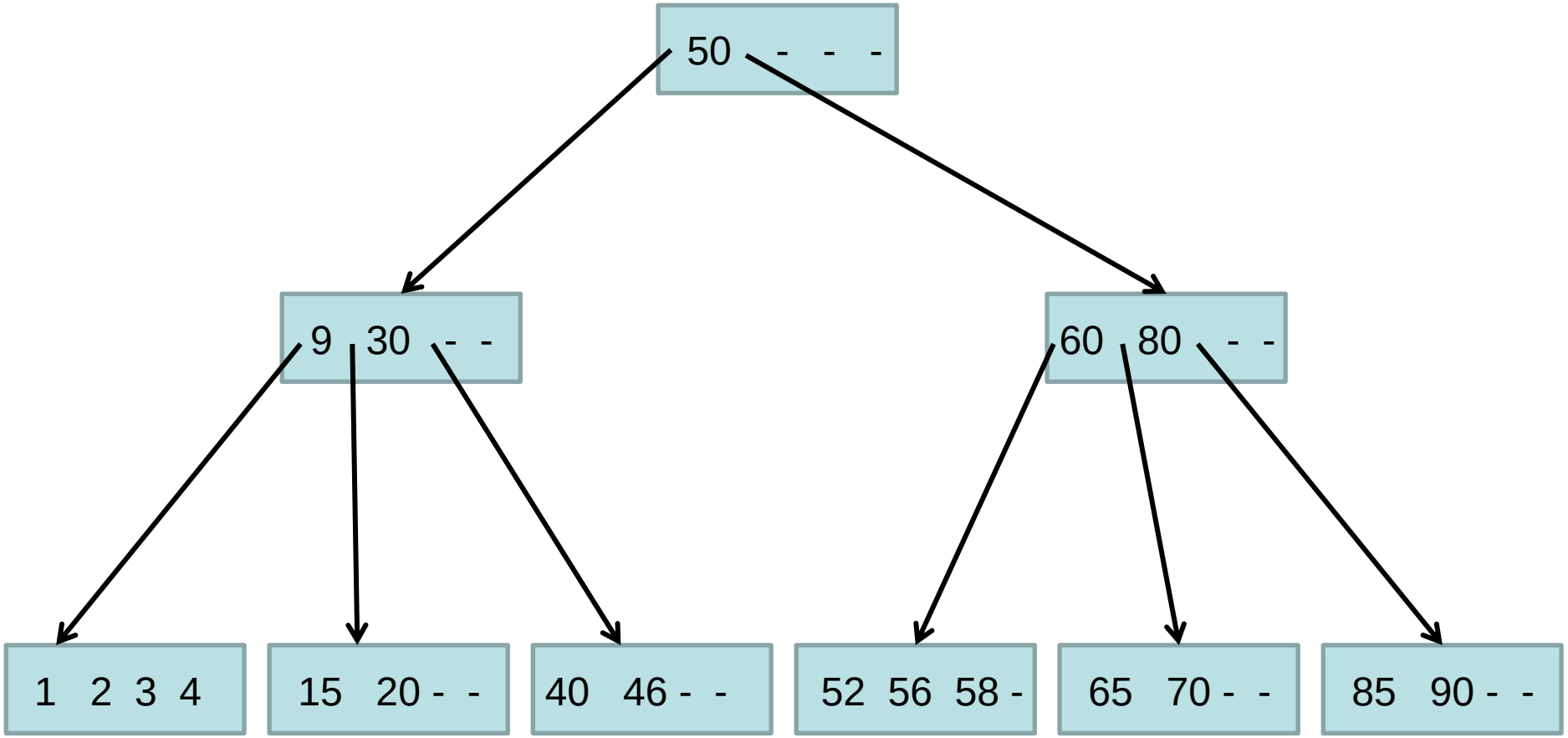
Concatenar P e Q.

Fazer a cisão de P e Q.

OBS. A redistribuição não é propagável

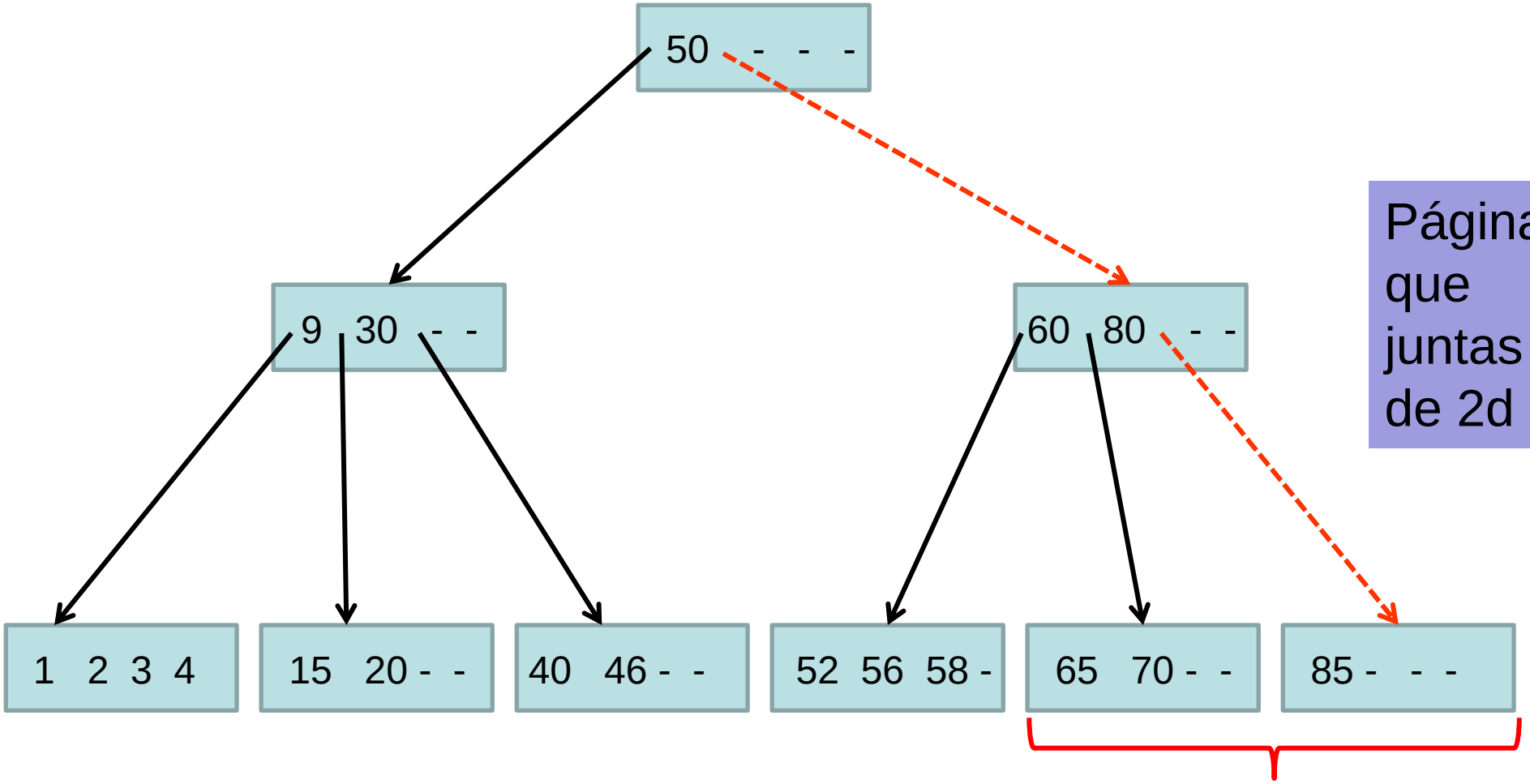
# Exercício

2. Considerando a árvore B de ordem 2 abaixo, remover a chave 90.



# Exercício

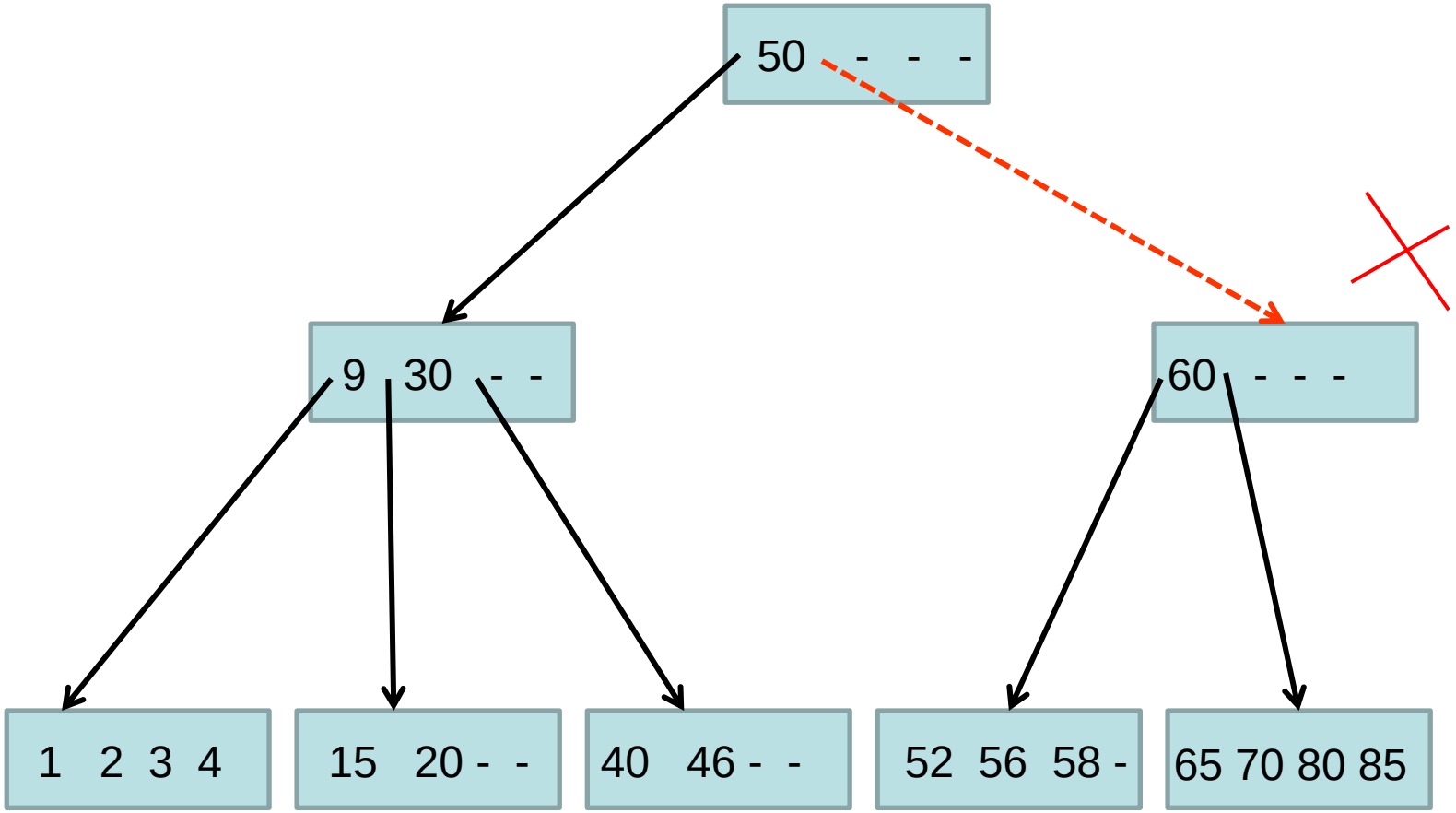
2. Considerando a árvore B de ordem 2 abaixo, remover a chave 90.




Páginas irmãs que possuem juntas menos de 2d chaves

# Exercício

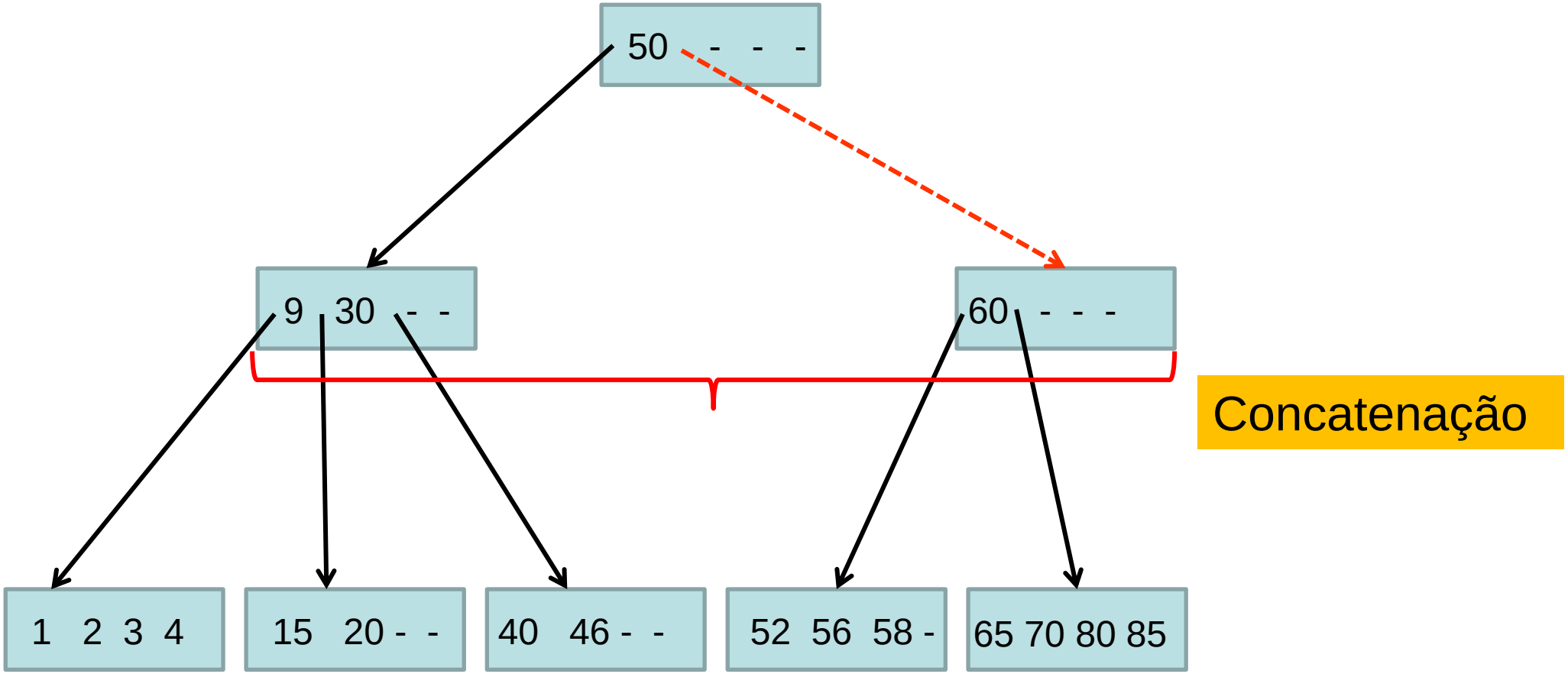
2. Considerando a árvore B de ordem 2 abaixo, remover a chave 90.




Concatenação

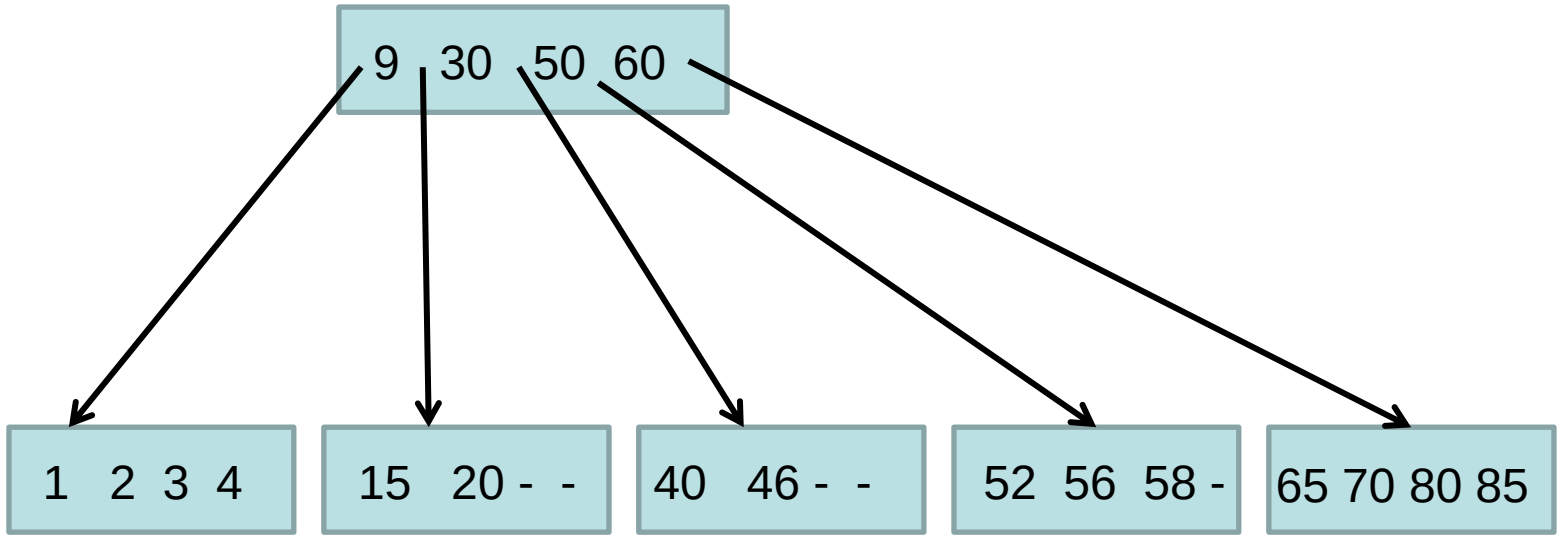
# Exercício

2. Considerando a árvore B de ordem 2 abaixo, remover a chave 90.



# Exercício

2. Considerando a árvore B de ordem 2 abaixo, remover a chave 90.



# Custo de Busca, Inserção e Remoção

Considere

Fmin – mínimo número de páginas acessadas

Fmax – máximo número de páginas acessadas

Emin – mínimo número de páginas escritas

Emax – máximo número de páginas escritas

# Custo de Busca, Inserção e Remoção

## BUSCA

Custo mínimo quando se encontra a chave na raiz.
**Fmin = 1**

Custo máximo quando se encontra a chave em uma folha.
**Fmax = *h***

Não há escrita de páginas
**Emin = Emax = 0**

# Custo de Busca, Inserção e Remoção

INSERÇÃO

Melhor caso quando nenhuma cisão é requerida.
**Fmin = *h*; Emin = 1**

Pior caso quando todas as páginas no caminho entre a raiz e a folha passam pelo processo de cisão.
**Fmax = *h*; Emax = 2*h* + 1**

# Custo de Busca, Inserção e Remoção

REMOÇÃO

Melhor caso quando nenhuma concatenação ou redistribuição são requeridas e a chave se encontra em uma folha.
**Fmin = *h*; Emin = 1**

Pior caso quando ocorre concatenação em todas as páginas do caminho, exceto no dois primeiros níveis. A página filho resulta em um número insuficiente de chaves sendo necessária uma redistribuição.

# Custo de Busca, Inserção e Remoção

REMOÇÃO

Há um total de h – 2 páginas escritas devido à concatenação e ainda 3 páginas escritas como resultado da redistribuição.

**Fmax = 2*h* – 1**

(*h* para baixo, *h* – 1 para cima olhando a página irmã.

**Emax = h+1**

# Exercícios em Árvore B

1. Fazer o algoritmo da cisão em uma árvore B de ordem d.

2. Fazer o algoritmo da concatenação em uma árvore B de ordem d.

3. Fazer o algoritmo da redistribuição em uma árvore B de ordem d.